

ARCHITETTURA 2

METODOLOGIA DI CLOCKING

- la **metodologia di clocking** definisce quando i segnali possono essere **letti** e quando possono essere **scritti**
- è importante specificare il **tempo delle letture e delle scritture**, perché se un segnale è scritto nello stesso momento in cui è letto, il valore letto potrebbe corrispondere al vecchio valore, a quello nuovo o addirittura ad un valore a cavallo tra i due

DEF.

La **metodologia di clocking** è l'approccio usato per determinare quando il *segnale* dell'informazione data in input è *stabile e valido* relativamente al clock.

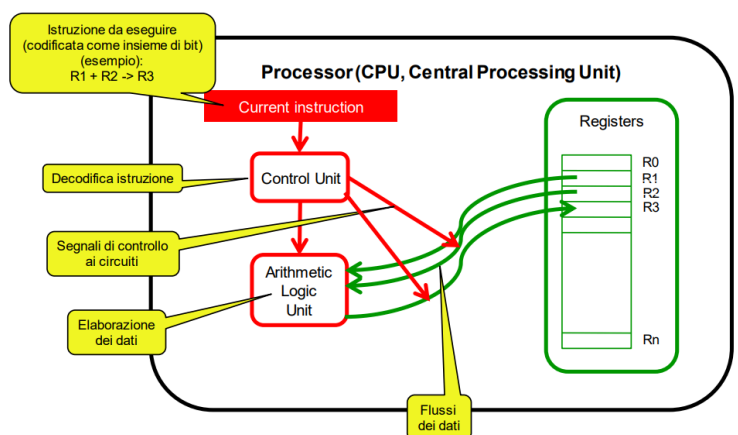
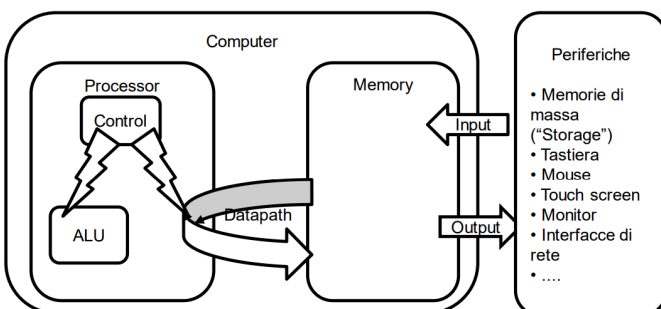
Edge –Triggered Clocking Methodology

- Una **edge-triggered metodologia di clocking** permette di aggiornare qualsiasi valore, salvato in un elemento di logica sequenziale, solo sul fronte di salita o di discesa del clock.
- gli input sono valori che sono stati scritti in un precedente ciclo di clock, mentre gli outputs sono valori che possono essere usati in un successivo ciclo di clock

DEF.

Una **edge-triggered metodologia di clocking** è uno schema di clocking in cui tutti i cambiamenti di stato avvengono sul fronte di salita o di discesa del clock.

COME LAVORA UN COMPUTER



Registri CPU:

- dentro il processore
- pochi (solitamente 32)
- veloci (coincide con il clock = 10^9 Hz)
- costosi
- quelli che di solito vengono chiamati **registri**

Memoria:

- grande (1GB)
- relativamente lenta (alcuni cicli di clock)
- mediamente costosa
- insieme di registri chiamati "**celle**" o "**locazioni**"

Memoria di massa ("storage"):

- periferica
- molto grande (da 100GB a 1TB)
- lenta (millisecondi)
- poco costosa
- persistente

FILOSOFIE DI PROGETTO DELLA CPU → RISC E CISC

★ **CISC → Complex Instruction Set Computing**

- un'architettura di tipo CISC è caratterizzata da un set di istruzioni costituito da istruzioni complesse (più tipi di istruzione che permettono di fare più cose) (che facilitano il lavoro al programmatore) ma allo stesso tempo molto più complicate da implementare dal punto di vista circuitale.
- i programmi sono relativamente **più corti** in quanto le istruzioni complesse permettono di "minimizzare" la quantità di istruzioni assembly necessarie e di conseguenza è necessaria **meno memoria RAM** per memorizzare le istruzioni.
- il compilatore traduce direttamente le istruzioni complesse in codice macchina: alla **complessità circuitale** "pagata all'inizio" corrisponde una maggiore **velocità di traduzione**.
- l'**esecuzione** delle istruzioni è più lenta a causa della complessità del circuito (i segnali devono passare da più porte e collegamenti)

★ **RISC → Reduced Instruction Set Computing**

- un'architettura di tipo RISC è caratterizzata da **un ridotto set di istruzioni semplici** (add, mux, sub...) e pertanto più facili da implementare a livello circuitale.
- i **programmi** sono **più lunghi** in quanto è necessario utilizzare più istruzioni (semplici) per scrivere programmi complessi e quindi di dimensione maggiore.
- esecuzione delle **istruzioni è più veloce** grazie alla semplicità del circuito (devono passare attraverso meno porte)

MIPS32

MIPS32 è un'architettura che implementa un set di istruzioni di tipo **RISC**.

- 32 registri, 32 bit
- istruzioni in 32 bit che rappresentano delle operazioni:
 - manipolazione di dati **solo** sui **registri**
 - trasferimento di **dati** tra *memoria e registri*
 - alterazione del flusso di controllo (salti)
- tre formati delle istruzioni:
 - 1) R-type
 - 2) I-type
 - 3) J-type

→ le istruzioni RISC sono sempre di 32 bit (4 byte) = 1 parola in MIPS

ASSEMBLY

- **linguaggio macchina** → in binario
 - linguaggio direttamente comprensibile dal calcolatore
 - attività di *programmazione più lunga*
 - facile commettere *errori*
- **linguaggio assembler** → assembly
 - rappresentazione simbolica del linguaggio macchina
 - più comprensibile del linguaggio macchina (usa simboli, non sequenze di bit)
 - tradotto dall'assemblatore in linguaggio macchina
 - dalla forma simbolica dell'istruzione macchina al corrispondente formato binario
- **linguaggio di alto livello**
 - tradotti dal compilatore in assembler → assemblatore, lo strumento non il linguaggio assembly

LINGUAGGIO DI ALTO LIVELLO

Vantaggi

- notazione vicina al linguaggio corrente e alta notazione algebrica → maggiore espressività e leggibilità
- incremento della produttività
 - la programmazione è svincolata dalla conoscenza dei dettagli architetturali della macchina utilizzata
- indipendenza dalle caratteristiche dell'architettura (processore) su cui il programma sarà eseguito (portabilità)
 - ideati non per essere compresi direttamente da macchine reali, ma da macchine astratte in grado di effettuare operazioni di più alto livello rispetto alle operazioni dei processori reali
- permettono l'uso di librerie di funzionalità già scritte (riusabilità del codice)

LINGUAGGIO ASSEMBLER

Vantaggi

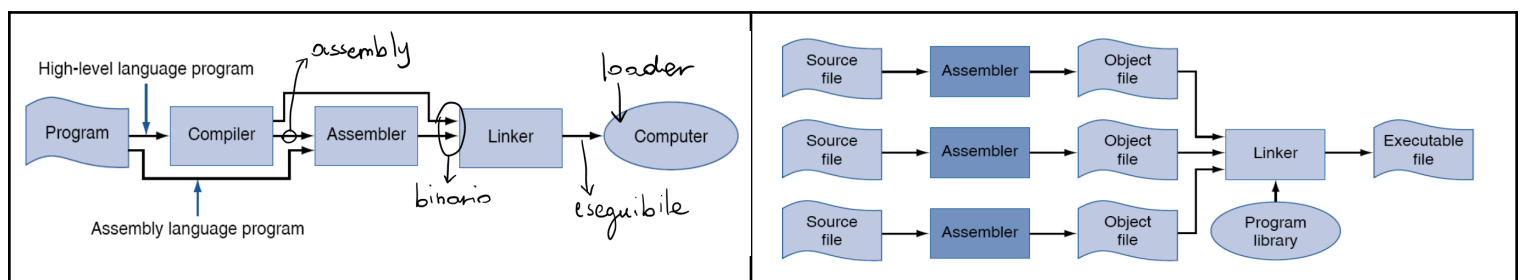
- ➔ la dipendenza dall'architettura del calcolatore permette:
 - velocità e dimensione più ridotta
 - ottimizzare le prestazioni → maggior efficienza
 - programmi (potenzialmente) più compatti
 - massimo sfruttamento delle potenzialità dell'hardware sottostante
 - molto importante per programmare controller e processi e macchinari (es. real-time), o per apparati limitati (es. embedded computer, portatili)

Svantaggi

- ➔ della programmazione
 - minore espressività (es. strutture di controllo limitate)
 - necessario conoscere i dettagli dell'architettura
 - mancanza di portabilità su architetture diverse
 - difficoltà di comprensione
 - lunghezza maggiore dei programmi

CATENA PROGRAMMATIVA

- la **catena programmatica** è tutto ciò che permette di eseguire un programma di alto livello in codice macchina, quindi tutto quello che sta in mezzo tra il codice di alto livello e il codice macchina



FORMATO DELLE ISTRUZIONI

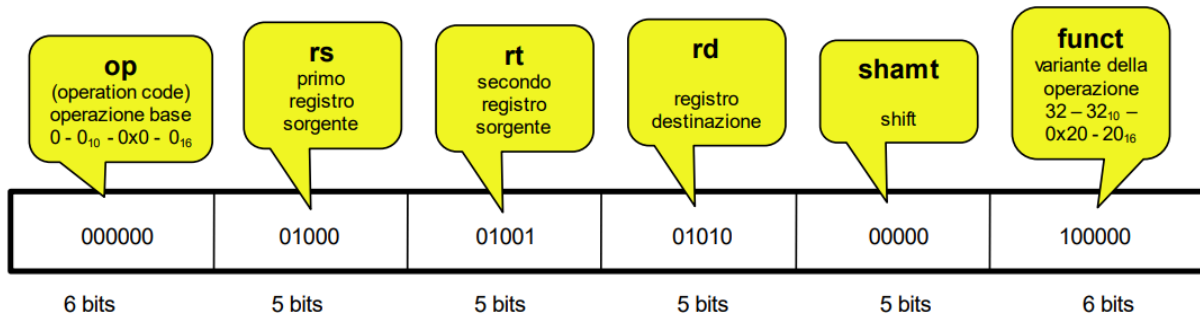
Le istruzioni assembly possono essere suddivise in **tre formati**:

- 1) R-type
- 2) I-type
- 3) J-type

R-TYPE

Esempio di istruzione: add

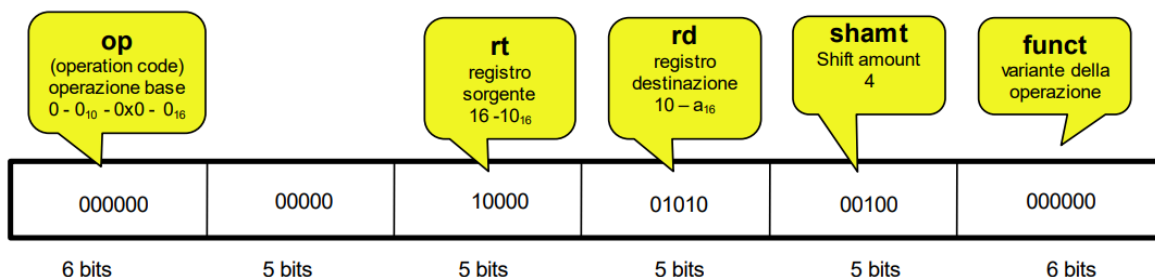
- “somma i contenuti del registro 8 e del registro 9 e metti il risultato nel registro 10”



- **OP** → **operation code**, indica il tipo di operazione (dal bit 31 al bit 26)
- **rs, rt** → sono i registri dei due operandi
- **rd** → registro in cui viene salvato il risultato
- **shamt** → **shift amount**, solitamente sono bit inutili sono utili solo per lo shift amount
- **funct** → sono i bit meno significativi, indicano una sotto operazione
- ➔ il formato simbolico assembly è un “**linguaggio sorgente**” → un programma è un testo
 - ◆ **add \$10, \$8, \$9**
 - codici operativi simbolici
 - nomi simbolici di registri (\$xxx)
 - nomi simbolici di variabili (che diventeranno indirizzi)
 - !!! attenzione all'**ordine** degli operandi, perché l'ordine con cui indico il risultato nella **memoria** non è lo stesso con cui lo indico nel codice / **istruzione assembly** !!!
- ➔ un programma **assemblatore** traduce da **codice sorgente** a **codice eseguibile**
- ➔ quello che viene caricato in **memoria** ed eseguito è sempre l'eseguibile (**codice macchina**)
- ➔ istruzioni R-type → **specifico** i registri

SHIFT LEFT

- “shift di 4 bit a sinistra il contenuto del registro 16 e metti il risultato nel registro 10”



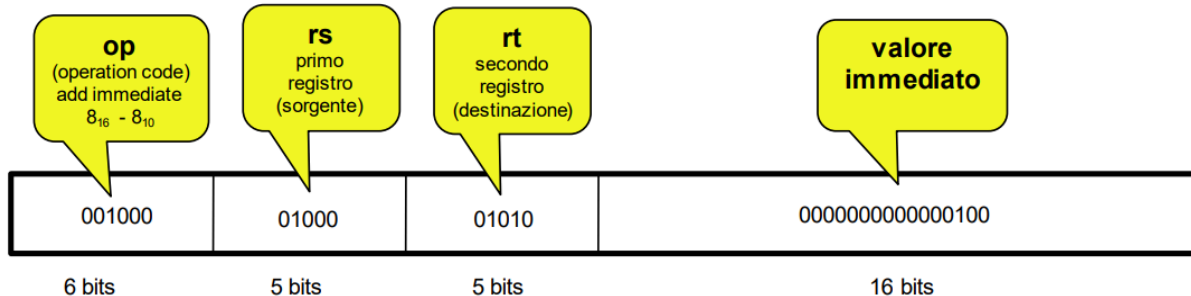
- se rt contiene inizialmente 9 → 0000 0000 0000 0000 0000 0000 1001 = 9₁₆
- dopo l'esecuzione rd contiene 144 → 0000 0000 0000 0000 0000 0000 1001 0000 = 90₁₆
- ASSEMBLY → **sll \$10, \$16, 4**
- OPPURE → **sll \$t2, \$s0, 4** → **SHIFT LEFT LOGICAL**

- ★ **shift left 1** → devo *spostare* verso **sinistra** di **1 posizione** i bit che compongono il numero da shiftare e aggiungere uno zero alla *prima posizione lasciata vuota a destra, togliendo il primo bit in testa a sinistra*
- ★ se il **nuovo MSB** è *diverso* dal precedente c'è **overflow**

I-TYPE

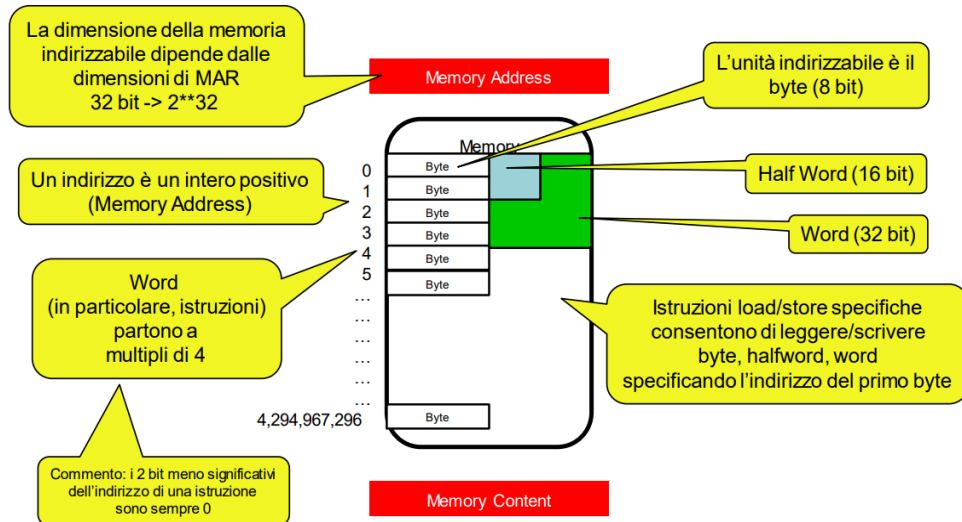
Esempio di istruzione: add immediate – **addi**

- l'**immediato** sono **16 bit** contenuti nell'**istruzione**, in cui è contenuto il **valore immediato** del **secondo operando**, espressi in **CA2**
 - sono messi in relazione con un registro di 32 bit → es. somma tra un immediato di 16 bit e un registro di 32



- **valore immediato** → il secondo operando non è in un registro ma nell'istruzione
 - ASSEMBLY
 - ◆ **addi \$10, \$8, 4**
 - dovendo fare una somma tra operandi a 32 bit e a 16 bit (immediate) la macchina estende il segno a sinistra → il MSB dell'immediato viene esteso finché non si ottiene un operando a 32 bit
 - non tutte le istruzioni formato I sono aritmetiche, altre sono **salti**
 - il range di valori immediati che si può esprimere con 16 bit in CA2 è $(-32768 \leq \text{valore} \leq 32767)$, quindi se serve un valore più grande va gestito a livello programmatico

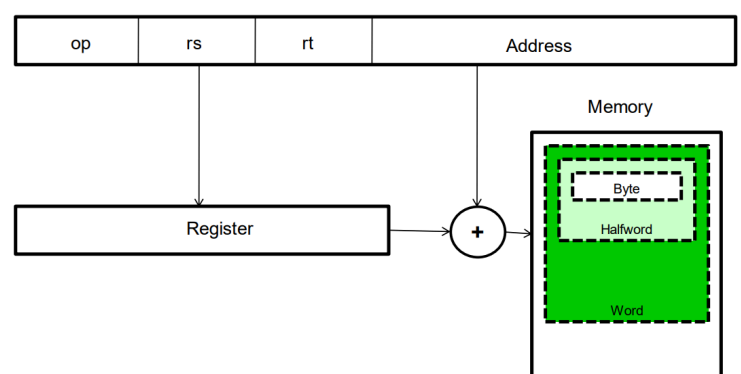
INDIRIZZAMENTO DELLA MEMORIA



La **memoria** opera su **byte** tramite l'utilizzo di **word**, parole → a blocchi di **4 byte = 32 bit**

BASE ADDRESSING – MODALITA' DI INDIRIZZAMENTO BASE / OFFSET

- caricamenti:
 - load byte 8 bit
 - load halfword 16 bit
 - load word 32 bit



ISTRUZIONE LOAD → prelievo di dati

→ collegamento sempre tra memoria e registri

Obiettivo → caricare in un registro del Register File il valore letto da un indirizzo di memoria calcolatoCosa dovrebbe **specificare** l'istruzione:

- 1) **OpCode**
- 2) registro di **destinazione**
- 3) **indirizzo** di memoria il cui **contenuto** viene **copiato** nel **registro di destinazione** (del Register File)

Problema → quanti bit occorrono?

- la memoria è grande (MIPS: 2^{32} locazioni)
- per specificare un indirizzo occorrono 32 bit...
- ... questi bit non ci stanno in un'istruzione di 32 bit

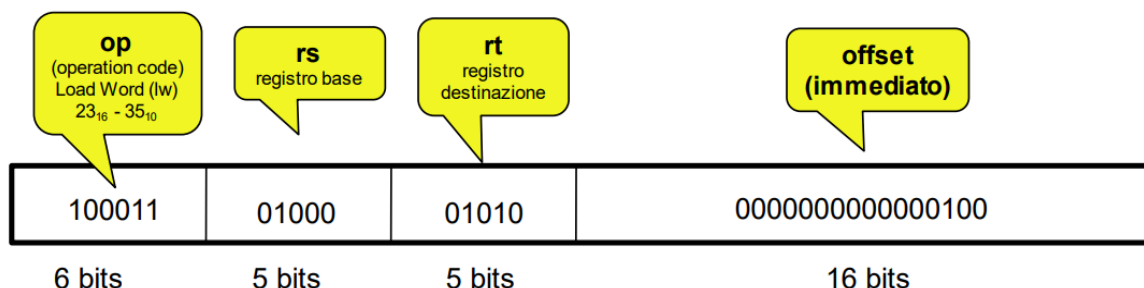
Soluzione → usare un formato **I-type**

- **I-type** → opcode, 2 registri, 1 immediato
- un registro è la **destinazione**
- un registro contiene un **indirizzo base** (a cui dobbiamo sommare l'offset per calcolare l'indirizzo da cui leggere in memoria)
- il valore immediato è interpretato come **spiazzamento (offset)** e rappresenta il numero di celle da saltare per allineare l'indirizzo base all'indirizzo in memoria
- l'indirizzo a cui accedere in memoria è calcolato come (**base + offset**)

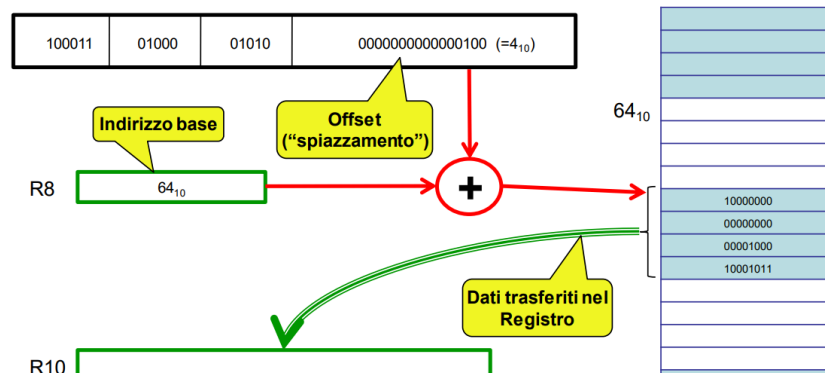
ISTRUZIONE LOAD WORD

- non c'è moltiplicazione dell'offset, viene sommato

Es. "carica nel registro 10 il contenuto della parola (32 bit) che è all'indirizzo di memoria ottenuto come somma del contenuto del registro 8 e dell'offset immediato 4"



- se rs contiene inizialmente 64_{10}
- l'**indirizzo di memoria** della word da caricare è 68_{10}
- in **rt** vengono caricati **32 bit** a partire dall'**indirizzo ottenuto**, qualunque sia il significato di quei 32 bit



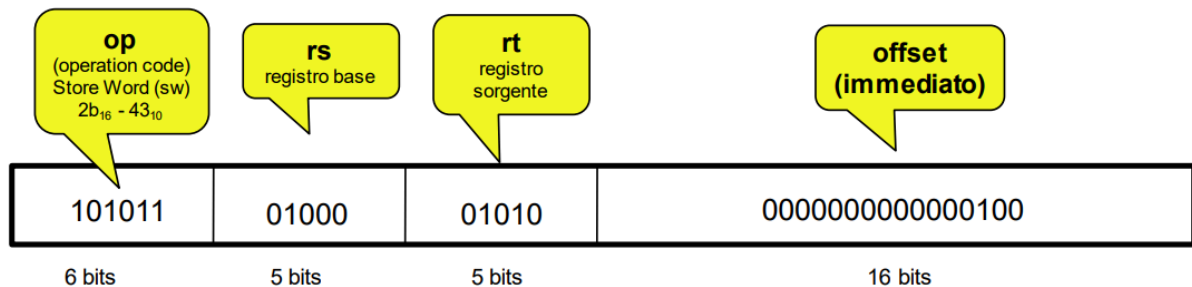
- **ASSEMBLY → lw \$10, 4(\$8)**

- \$10 indirizzo di destinazione → dove voglio caricare il dato nel Register File
- 4 offset → di quanto mi sposto
- \$8 indirizzo base → a cui va sommato l'offset per capire da quale registro prelevare il dato da salvare

→ vai a mettere nel registro \$10 contenuto nel Register File il valore contenuto in un indirizzo della memoria, tale indirizzo è calcolato sommando al registro base l'offset

ISTRUZIONE STORE WORD

Es: "memorizza il contenuto del registro 10 (32 bit) all'indirizzo di memoria ottenuto come somma del contenuto del registro 8 e dell'offset immediato 4"



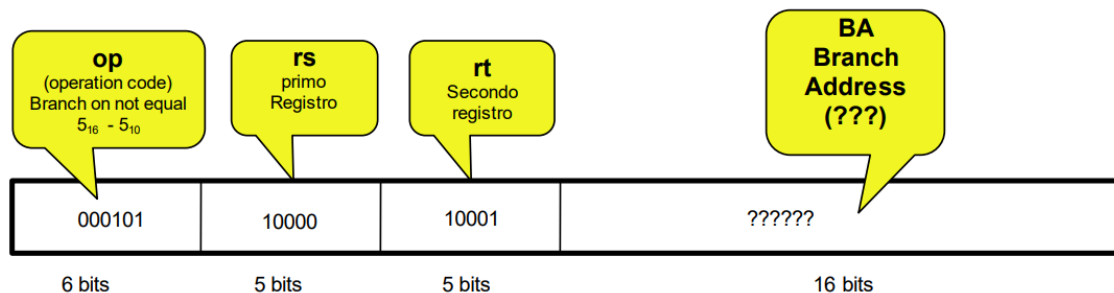
- **ASSEMBLY → sw \$10, 4(\$8)**

- rs = \$8 → contiene un indirizzo a cui viene sommato l'offset dove salveremo il nostro dato
- 4 spostamento
- \$10 → contiene il valore che vogliamo salvare in memoria

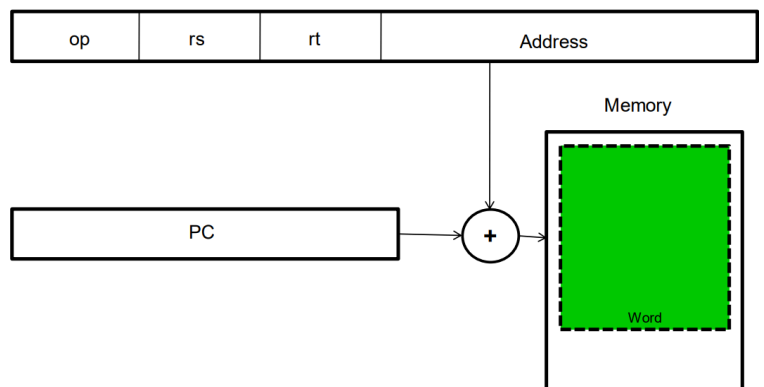
LOAD WORD	STORE WORD
carica nel registro 10 il contenuto della parola (32 bit) che è all'indirizzo di memoria ottenuto come somma del registro 8 e dell'offset immediato 4	memorizza il contenuto del registro 10 (32 bit) all'indirizzo di memoria ottenuto come somma del registro 8 e dell'offset immediato 4
lw \$10, 4(\$8)	sw \$10, 4(\$8)

ISTRUZIONE BRANCH → indico di quanto spostarsi

- "salta a branch address se il contenuto del registro 16 (rs) è diverso dal contenuto del registro 17 (rt)"



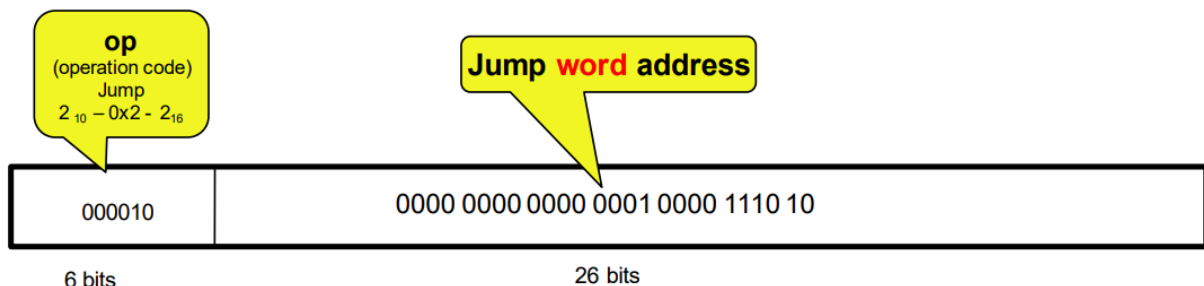
- 16 bits – **Branch Address** → *numero di bit* che vado a sommare ad un *registro* speciale che è il **Program Counter** → questi 16 bit indicano il numero di istruzioni da saltare, e se sommati all'indirizzo contenuto nel PC ti permettono di *ricavare l'indirizzo dell'istruzione* su cui si vuole "atterrare"
- ASSEMBLY → **bne \$16, \$17, Exit**
- dove si può saltare? → **valore immediato** – $2^{15} \leq BA < 2^{16}$ → 16 bit, intervallo di numeri in CA2, numero negativo vado indietro / numero positivo vado avanti
 - 1) usare un **registro base R** → *indirizzo di salto* = $R + BA$
 - 2) usare il **Program Counter** come registro base
- ★ se **CA** è l'indirizzo dell'istruzione corrente, si salta a **CA + 4** (*prossima istruzione*) + **BranchAddr*4**
 - l'immediato è **BranchAddr** e viene *aggiunto* al **registro PC**
 - **BranchAddr*4** → *BranchAddr* indica il numero di istruzioni da saltare, e viene **moltiplicato per 4** per raggiungere il primo byte che indirizza l'inizio di una nuova istruzione
 - quindi ci spostiamo di **4 byte in 4 byte** perché ogni *istruzione* è memorizzata su 4 byte e **non** possiamo saltare a **metà word**, ma dobbiamo saltare all'*inizio*
 - *BranchAddr* rappresenta il **numero di word** da saltare rispetto all'*indirizzo memorizzato nel PC*
 - dopo aver *prelevato dalla memoria un'istruzione*, il Program Counter punta all'istruzione successiva, quindi **contiene CA + 4**
 - ampiezza del salto: $\pm 2^{15}$ rispetto a **PC**
- adeguata in molti casi (**loop, if...**)
- **branch on equal** → beq
- **branch on greater than zero**
- **branch on less than zero**



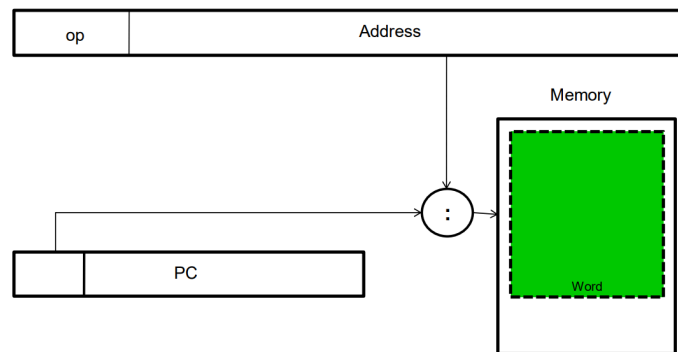
J-TYPE

ISTRUZIONE JUMP → indico l'indirizzo preciso a cui saltare

- "salta all'istruzione il cui indirizzo è $00010e8_{16}$ "



- carica in PC **0000 0000 0000 0001 0000 1110 1000**
 - poiché le istruzioni **devono** essere allineate al word (responsabilità di chi scrive e carica i programmi in memoria → in pratica *assembler* e *loader*)
- quindi con **26 bit** si indirizzano **2^{26} byte di memoria programma** oppure **2^{26} word**
- ai bit shiftati di 2 si mettono in testa i 4 bit più significativi del PC (già incrementato) → [31 – 28]
- **PC – Program Counter** → contiene l'*indirizzo dell'istruzione successiva* da eseguire



DATAPATH

Il **datapath** è il meccanismo che interconnette le varie **unità funzionali** della CPU al fine di costruire una *struttura* in grado di eseguire le istruzioni macchina che costituiscono i programmi.

Componenti generali del datapath:

- i **multiplexor** implementano la logica con cui si sceglie quale tra le sorgenti relazionare sulla base di linee di controllo prese dai campi dell'istruzione in esecuzione
- l'**unità di controllo** prende in input i bit dell'istruzione in esecuzione e imposta, sulla base di essi, le linee di controllo per i multiplexor e per pilotare le unità funzionali
- gli **elementi combinatori** sono *elementi che eseguono operazioni* come un'ALU
- gli **elementi di stato** sono *elementi di memoria* come i registri o la memoria
 - un elemento di stato ha almeno due input e un output
 - ➔ gli input necessari sono il valore dell'informazione da salvare nell'elemento e il clock che determina quando il valore del dato da scrivere è stato scritto
 - ➔ l'output di un elemento di stato fornisce il valore che era stato scritto in un ciclo di clock precedente
 - sono D-Flip Flops, memoria, registri
 - un elemento di stato può essere **letto in ogni momento**
 - le componenti logiche che contengono lo stato sono anche chiamate **sequenziali**, perché i loro output dipendono sia dagli input sia dal contenuto dello stato interno

Le **unità funzionali** sono:

- **MEMORIA** → contiene **istruzioni e dati** salvati in **locazioni di memoria**.
 - Permette la lettura o scrittura di *locazioni di memoria*, ciascuna individuata da un **indirizzo**.
 - Sulla base dell'indirizzo in input, restituisce in output l'istruzione o il dato letto.
 - **REGISTER FILE** → è un elemento di stato che consiste in una **collezione di 32 registri** in cui ogni registro può essere **letto o scritto** specificando il numero (binario) del registro grazie al quale si può accedere all'informazione contenuta in tale registro.
 - ➔ contiene **32 registri** che vengono utilizzati per *l'esecuzione delle istruzioni*.
- Input:
- indirizzi di **2 registri da leggere**
 - indirizzo di **1 registro da scrivere**
 - **segnale** che **abilita la scrittura**
 - **dato** da *scrivere* nel **registro specificato** in input
- ➔ Restituisce in **output** il contenuto dei 2 registri letti e scrive, se il *segnale di scrittura* è *attivo*, il dato in input all'indirizzo specificato come altro input.
- ➔ la **scrittura** di un registro, a differenza della **lettura** che non deve essere specificata, **deve essere esplicitamente indicata asserendo il segnale di controllo della write**.

- da ricordare che la scrittura è edge-triggered, quindi tutti gli input di una write devono essere validi sul fronte di salite o discesa del clock
- **ALU** → si occupa di effettuare operazioni aritmetico-logiche.
 - **per istruzioni R-type** → esegue operazioni aritmetico-logiche su due operandi in funzione del `function_code` indicato dai 6 bit meno significativi dell'istruzione
 - **per istruzioni di accesso a memoria** → calcola l'indirizzo di memoria cui accedere
 - **per istruzioni branch** → confronta i due registri in input
 - **per tutte le istruzioni** → incrementa il PC ($PC+4$) e calcola il valore dell'indirizzo di branch (che sarà usato solo nel caso di istruzioni di branch)
- **SIGN-EXTEND** → opera l'**estensione di segno aritmetica** da **16 a 32 bit**. Utilizzata per il calcolo dell'*indirizzo di branch*, *lw* e *sw*.
- **SHIFT-LEFT-2** → opera uno **shift a sx** dei *bit in input* (come se *moltiplicasse per 4*).
 - I **bit** presi in **input** sono quelli dell'**offset (16 bit finali)** dell'istruzione di branch che ci permettono di calcolare a quale istruzione effettuare il salto.
 - è utilizzata per il calcolo degli indirizzi di **jump** e **branch**, che devono essere **allineati al word** e perciò devono essere multipli di 4 (per ottenere un salto che ti porta all'inizio di una nuova word e non a metà di un'istruzione).

IL CICLO DI FETCH

Il datapath svolge periodicamente, istruzione per istruzione, il così detto **ciclo di Fetch**, composto da **3 fasi**:

- 1) **FETCH** → viene **prelevata dalla memoria l'istruzione da eseguire** e successivamente il puntatore alla memoria (**Program Counter**) viene **incrementato** in modo che punti all'istruzione successiva
- 2) **DECODE** → usa i bit dell'istruzione macchina per **pilotare** il datapath in modo che *si prepari ad eseguire l'istruzione*. L'*unità funzionale* che pilota e quindi "apre e chiude" i collegamenti nel datapath è la **Control Unit (CU)**.
 - è dunque la CU che ad esempio "sceglie", prendendo in input l'istruzione macchina, se selezionare come operazione dell'ALU una somma oppure una differenza.
- 3) **EXECUTE** → vengono effettuati i passi necessari per l'**esecuzione** dell'istruzione

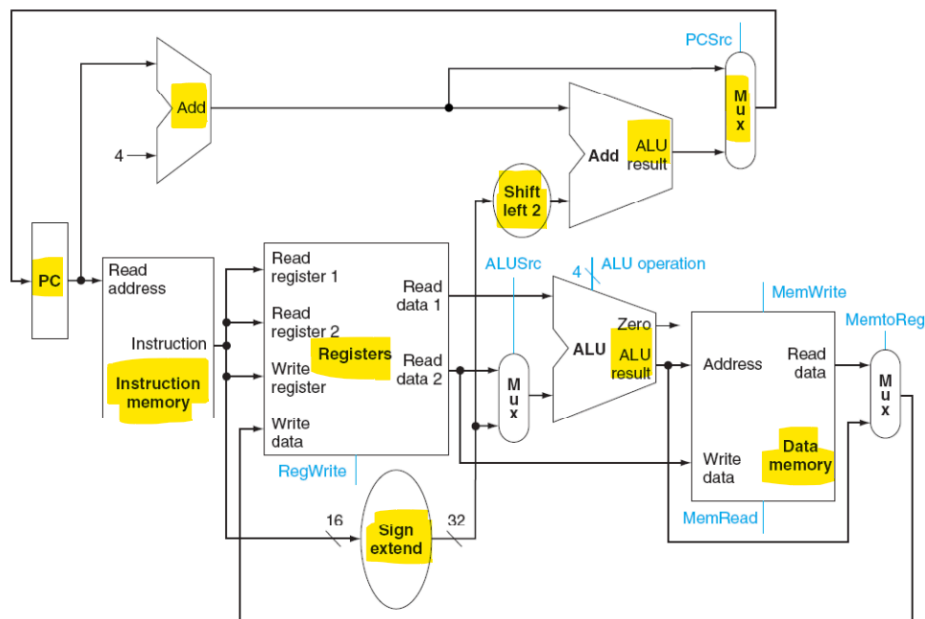
DATAPATH MULTICICLO E SINGOLO CICLO

Per costruire un datapath è inoltre necessario distinguere due *diverse architetture*, **singolo ciclo** o **multiciclo**. Queste due alternative presentano *sostanziali differenze*:

1) **SINGOLO CICLO**

- l'istruzione, in tutte le sue 3 fasi, viene completamente eseguita in un **unico ciclo di clock**.
- Dal momento che le esecuzioni delle diverse istruzioni impiegano più o meno tempo l'una rispetto all'altra è necessario avere un ciclo di clock **sufficientemente lungo** da far rientrare tutte le possibili istruzioni.
 - A tal proposito, è intuibile pensare che tale soluzione comporti un **notevole spreco di tempo** in quanto l'esecuzione dell'**istruzione più corta possibile** terminerebbe ben **prima della fine del ciclo di clock**.
 - ★ Dunque le istruzioni più **veloci** sono **rallentate**.
- più **semplice da implementare** ma comporta allo stesso tempo la duplicazione di alcune unità funzionali.
 - Infatti, dal momento che in un singolo ciclo di clock è possibile utilizzare ciascuna unità funzionale **una e una sola volta**, è necessario avere più "*copie*" della *stessa unità* per poter utilizzare in momenti diversi la stessa funzionalità.

- Ad esempio, nel caso dell'operazione di **add**, avrò bisogno di un circuito sommatore sia per *incrementare il program counter* nella fase di fetch (così da puntare all'istruzione successiva), sia per effettuare l'effettiva *operazione di somma*.
- Un altro esempio è la **memoria**: nel caso di operazioni di tipo **sw**, sarà necessario *accedere alla memoria* sia per *recuperare l'istruzione* che per *salvare in memoria il dato* come fine ultimo dell'operazione di "store". Anche in questo caso sarà necessario *accedere alla memoria per più volte* in un solo ciclo di clock e pertanto sarà necessario duplicare tale unità funzionale.



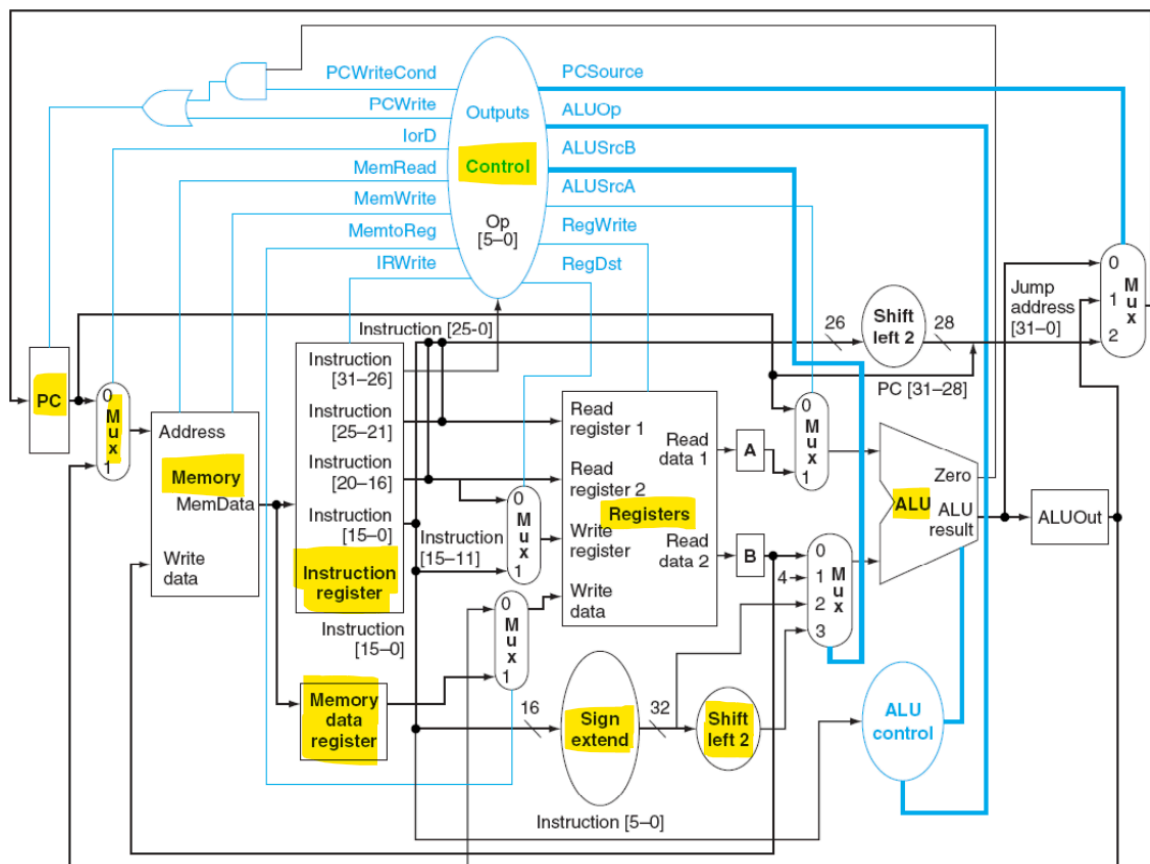
2) MULTICICLO

- l'esecuzione dell'istruzione viene **spezzettata in più cicli di clock** di inferiore durata.
- Il **numero di cicli di clock necessari** dipende dal **tipo di istruzione** e possono essere 3, 4 o 5.
 - In questo modo le **istruzioni più brevi** (quelle che richiedono solo 3 cicli di clock) non **sprecano** inutilmente **tempo** di utilizzo della CPU.
 - ★ Il datapath multiciclo permette quindi un **utilizzo più efficiente della CPU** che riduce al minimo gli sprechi.
- **Non è necessario duplicare le unità funzionali** perché, in cicli di clock differenti, possono essere riutilizzate.
 - La ALU viene dunque utilizzata sia per le *operazioni aritmetico-logiche* che per il *calcolo di indirizzi di branch e jump*.
 - La memoria, a sua volta, viene utilizzata sia per *leggere le istruzioni* che per *leggere o scrivere dati*.
 - Ad esempio l'istruzione di **add** viene eseguita in **4 cicli di clock**: i primi 2 per fetch e decode (*uguali per tutte le istruzioni*) e un terzo ciclo di clock per il calcolo della somma e un quarto ciclo per il salvataggio del risultato sul registro indicato. L'ALU (che svolge operazioni aritmetico-logiche come in questo caso la somma) viene adoperata sia nel primo ciclo di clock ($PC+4$), sia nel secondo (calcolo dell'indirizzo di branch), che nel terzo (somma degli operandi), viene pertanto *riutilizzata*.
- Per rendere possibile l'esecuzione di un'istruzione in più cicli di clock, è necessario avere dei **registri** che, tra un ciclo di clock e l'altro, **tengono memoria dei risultati intermedi**.

- Così facendo questi ultimi possano essere **utilizzati nel successivo colpo di clock** per *riprendere l'esecuzione dell'istruzione*.
- Creano quindi **separazione nel tempo**, fotografando i valori del ciclo di clock precedente.

→ I registri aggiuntivi sono:

- **IR** → **instruction register** → Memorizza i bit dell'**istruzione prelevata** (non il suo indirizzo!)
 - necessario per permettere di passare dalla Fetch alla Decode
 - i bit dell'istruzione vengono letti dalla memoria (*Fetch*) e appoggiati nell'**instruction register**, dal quale saranno poi passati all'unità di controllo del datapath (*Decode*)
- **MDR** → **memory data register** → **Memorizza** il dato letto dalla memoria **prima** che sia scritto nel registro destinazione del **Register File** (usato nell'istruzione load word).
- **A, B** → Memorizzano i valori degli **operandi** dati in input alla **ALU** provenienti dal register file
- **ALUout** → Memorizza il **risultato** prodotto dalla **ALU**.



REGISTRI DATAPATH MULTICICLO

- 1) **PC** → contiene i 32 bit che indicano l'indirizzo dell'istruzione da eseguire
- 2) **Instruction register** → contiene i 32 bit che corrispondono alla codifica dell'istruzione prelevata dalla memoria per l'esecuzione
- 3) **Memory Data Register** → contiene il dato letto da memoria prima della sua scrittura nel registro di destinazione (es. nel caso di esecuzione dell'istruzione load word)
- 4) **A e B** → contengono i valori dei registri letti dal Register File
- 5) **ALUOut** → contiene l'output dell'ALU

Multiplexor:

- **A** → seleziona l'indirizzo di memoria a cui accedere tra:
 - PC (nel caso si voglia leggere un'istruzione)
 - output della ALU (nel caso di esecuzione dell'istruzione load)
- **B** → seleziona il gruppo bit dell'IR che indica il registro in cui scrivere per i due tipi di istruzione I-type (IR[20:16]) e R-type (IR[15:11])
- **C** → seleziona la sorgente per il dato da scrivere in memoria tra
 - ALUOut (per istruzioni R-type)
 - MemoryDataRegister (per istruzioni load)
- **D** → seleziona il primo operando dell'operazione aritmetico-logica eseguita dalla ALU tra PC (per incremento +4) e registro A (per istruzioni R-type)
- **E** → seleziona il secondo operando dell'operazione aritmetico-logica eseguita dalla ALU
 - B (per istruzioni R-type)
 - 4 (per aggiornamento del PC)
 - sign-extended IR[15:0] (per istruzioni di accesso a memoria – lw /sw)
 - sign-extended 2-shifted (branch)
- **F** → seleziona il valore per l'aggiornamento del PC
 - output dell'ALU: PC + 4
 - contenuto di ALUOut → 2 shifted 32-bit beq field
 - jump target address (IR[25:0] shifted left 2 bits e concatenato con PC + 4[31:28])

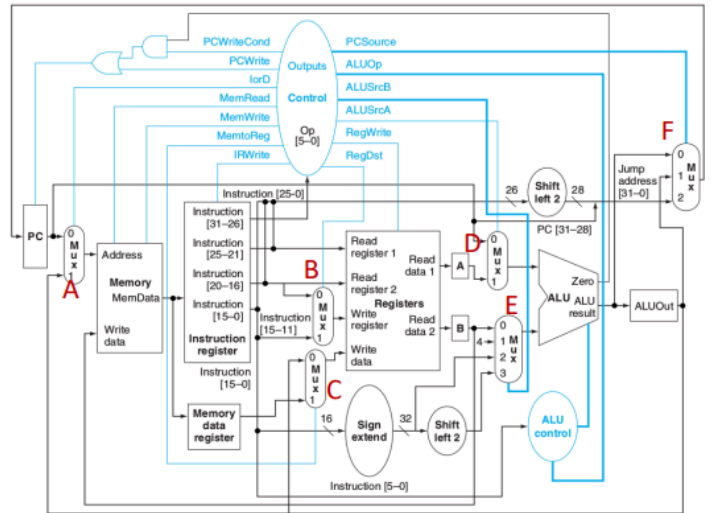


FIGURE 5.28 The complete datapath for the multicycle implementation together with the necessary control lines. The con-

Actions of the 1-bit control signals

Signal name	Effect when deasserted	Effect when asserted
RegDst	The register file destination number for the Write register comes from the rt field.	The register file destination number for the Write register comes from the rd field.
RegWrite	None.	The general-purpose register selected by the Write register number is written with the value of the Write data input.
ALUSrcA	The first ALU operand is the PC.	The first ALU operand comes from the A register.
MemRead	None.	Content of memory at the location specified by the Address input is put on Memory data output.
MemWrite	None.	Memory contents at the location specified by the Address input is replaced by value on Write data input.
MemtoReg	The value fed to the register file Write data input comes from ALUOut.	The value fed to the register file Write data input comes from the MDR.
IorD	The PC is used to supply the address to the memory unit.	ALUOut is used to supply the address to the memory unit.
IRWrite	None.	The output of the memory is written into the IR.
PCWrite	None.	The PC is written; the source is controlled by PCSource.
PCWriteCond	None.	The PC is written if the Zero output from the ALU is also active.

Actions of the 2-bit control signals

Signal name	Value (binary)	Effect
ALUOp	00	The ALU performs an add operation.
	01	The ALU performs a subtract operation.
	10	The funct field of the instruction determines the ALU operation.
ALUSrcB	00	The second input to the ALU comes from the B register.
	01	The second input to the ALU is the constant 4.
	10	The second input to the ALU is the sign-extended, lower 16 bits of the IR.
	11	The second input to the ALU is the sign-extended, lower 16 bits of the IR shifted left 2 bits.
PCSource	00	Output of the ALU ($PC + 4$) is sent to the PC for writing.
	01	The contents of ALUOut (the branch target address) are sent to the PC for writing.
	10	The jump target address ($IR[25:0]$ shifted left 2 bits and concatenated with $PC + 4[31:28]$) is sent to the PC for writing.

ESECUZIONE DELLE ISTRUZIONI IN UN DATAPATH MULTICICLO

Step name	Action for R-type instructions	Action for memory-reference instructions	Action for branches	Action for jumps
Instruction fetch	$IR \leftarrow \text{Memory}[PC]$ $PC \leftarrow PC + 4$			
Instruction decode/register fetch	$A \leftarrow \text{Reg}[IR[25:21]]$ $B \leftarrow \text{Reg}[IR[20:16]]$ $ALUOut \leftarrow PC + (\text{sign-extend}(IR[15:0]) \ll 2)$			
Execution, address computation, branch/jump completion	$ALUOut \leftarrow A \text{ op } B$	$ALUOut \leftarrow A + \text{sign-extend}(IR[15:0])$	if $(A == B)$ $PC \leftarrow ALUOut$	$PC \leftarrow \{PC[31:28], (IR[25:0], 2'b00)\}$
Memory access or R-type completion	$\text{Reg}[IR[15:11]] \leftarrow ALUOut$	Load: $MDR \leftarrow \text{Memory}[ALUOut]$ or Store: $\text{Memory}[ALUOut] \leftarrow B$		
Memory read completion		Load: $\text{Reg}[IR[20:16]] \leftarrow MDR$		

Le fasi di **Fetch** e **Decode** sono comuni a tutte le istruzioni

> FETCH

→ in breve ha 2 obiettivi:

- 1) l'istruzione viene prelevata dalla memoria
- 2) l'istruzione register viene modificato
- 3) il PC viene incrementato così che possa puntare all'istruzione successiva

→ tutti i passaggi:

Fase 1

- l'indirizzo contenuto nel PC deve fluire in memoria per poter leggere l'istruzione corrente
- dato che i dati che entrano in memoria possono provenire da due registri diversi:
 - 1) il Program Counter
 - 2) ALUOut
 occorre impostare a 0 il segnale di $lorD$ per poter selezionare il PC come indirizzo entrante in memoria → perché con tale indirizzo faremo riferimento ad una cella di memoria che contiene l'istruzione che deve essere eseguita
- l'istruzione viene letta dalla memoria all'indirizzo indicato dal PC grazie al segnale $MemRead$ impostato a 1 e al segnale $MemWrite$ impostato a 0 (perché non dobbiamo aggiornare nessun valore in memoria ma solo accedere ad una cella e leggere il contenuto)

Fase 2

- viene fatta fluire l'istruzione contenuta nella cella in memoria nell'**Instruction register**, grazie al segnale IRWrite settato a 1, dove viene "parcheggiata" in attesa di essere usata per la Decode

Fase 3

- dobbiamo incrementare il PC
- useremo la ALU a cui passeremo l'indirizzo corrente del PC e il valore di default 4 per eseguire la computazione e sommare 4 byte a tale indirizzo di modo da far puntare il PC all'istruzione successiva, e quindi aggiornarlo
- l'operando A dovrà far entrare il contenuto del PC non aggiornato, quindi ALUSrcA sarà impostato a 0
- l'operando B contiene il valore 4, che se sommato all'indirizzo corrente permette il salto all'indirizzo dell'istruzione successiva
- per impostare B a 4 il segnale ALUSrcB deve essere impostato a 01
- ALUOp specifica all'ALU il tipo di operazione da effettuare, che nella fase di Fetch deve essere un'addizione per aggiornare il PC → ALUOp = 00
- il risultato dell'ALU deve andare a sovrascrivere il contenuto del PC non aggiornato, quindi fluisce in un multiplexor che riceve 00 come segnale, che darà in output l'indirizzo del PC aggiornato
- tale indirizzo sarà salvato nel PC grazie al segnale PCWrite settato ad 1, che permette di sovrascrivere il vecchio indirizzo con il nuovo

➤ DECODE

→ in breve ha 3 obiettivi:

- 1) l'istruzione viene letta
- 2) i bit vengono passati dall'Instruction Register all'unità di controllo che pilota il datapath
- 3) viene fatto il calcolo dell'indirizzo di branch → istruzione non sempre utilizzata ma se dovessimo fare tale operazione solo quando necessario, l'architettura sarebbe molto più complicata a livello circuitale, quindi se non è indicata nell'istruzione viene fatta e ignorata

→ tutti i passaggi:

Fase 1

- i primi 6 bit dell'istruzione, "parcheggiata" in fase di Fetch nell'Instruction Register, vengono fatti fluire all'unità di controllo che poi va a pilotare il datapath
- i bit riservati agli **operandi** specificano i registri da leggere contenuti nel Register File (necessari in fase di EX) → stiamo preparando gli operandi
- vengono preparati gli operandi A e B che verranno poi eventualmente utilizzati dall'ALU nella fase di EX (nel caso si tratti di una R-Type)
 - i bit passati dall'Instruction Register al Register File permettono di preparare A e B con il contenuto dei registri operandi
- ★ RegWrite deve essere a 0 perchè in questa fase non deve essere possibile scrivere nel Register File

Fase 2

- i primi 6 bit dall'Instruction Register fluiscono nell'unità di controllo
- questi bit vengono utilizzati nel colpo di clock successivo quando si va ad effettuare la fase di EX
- perchè nel colpo di clock corrente l'unità di controllo è occupata a calcolare l'indirizzo di branch (gli operandi A e B sono preparati senza intervento dell'unità di controllo)

Fase 3

★ gli **indirizzi** nel PC e nella memoria sono **allineati al byte**

- quindi in memoria potremmo sostanzialmente andare a prendere **qualsiasi cella**, anche quelle che *non rappresentano l'inizio di una word*
- quindi **ogni cella della memoria** contiene un **byte**, per questo la quantità minima che si può indirizzare è un byte
- quindi si può leggere e scrivere come minimo un *blocco di 1 byte = 8 bit*
- la **memoria** è quindi divisa da celle **ciascuna da 1 byte** e ciascuna **ha il suo indirizzo**
- quando andiamo a recuperare le istruzioni dalla memoria durante l'operazione di **Fetch** andiamo a leggere un'intera parola, quindi un blocco di 4 byte → **NON** potrà essere indirizzato un byte che **NON è l'inizio di una word**
 - è possibile farlo però si genererà un'**eccezione grave**
- per poter far sì che l'indirizzo che stiamo andando a calcolare per l'indirizzo di branch punti al primo byte dell'istruzione successiva, l'**offset viene moltiplicato per 4**, ovvero **shiftato per 2**
 - se un numero viene moltiplicato per 4 sarà per forza multiplo di 4 quindi di conseguenza si atterrerà **sul primo byte** della **word successiva**
- contemporaneamente occorre calcolare l'eventuale indirizzo di branch (evnt. perchè non sempre è usato)
- gli ultimi 16 bit di un'istruzione I-Type rappresentano i bit dell'offset (l'offset indica quante istruzioni si vuole saltare)
- in un'istruzione R-Type/J-Type (quindi qualsiasi altro tipo di istruzione che non sia una branch – anche una addi che è sempre un I-Type) *vengono comunque prelevati questi bit* per il calcolo dell'indirizzo di branch, anche se non sono dedicati a quello e quindi il risultato di tale calcolo **non verrà considerato**
- i **16 bit dell'offset** devono essere conformati al numero di bit con cui opera l'ALU, in quanto l'ALU esegue computazioni su **registri formati da 32 bit**.
- per questa ragione l'offset passa attraverso il **sign extend** che lo estende a 32 bit
 - se il MSB è 0 metterà tutti i bit mancanti a 0
 - se il MSB è 1 metterà tutti i bit mancanti a 1
- per calcolare l'indirizzo di branch dobbiamo **sommare all'indirizzo corrente** un certo numero di byte – chiamato **spiazzamento**
 - **indirizzo corrente** → contenuto nel PC – l'indirizzo nel PC è espresso **al byte**, ossia si riferisce all'inizio di una parola – al primo byte che ne identifica l'inizio = primi 8 bit
 - **spiazzamento** → sono i byte che separano l'istruzione corrente dall'istruzione a cui si desidera saltare → il totale delle word da saltare espresso in byte
- per ottenere il numero di istruzioni da saltare, quindi lo spiazzamento, occorre prendere i bit dell'offset e moltiplicarli per 4
- per moltiplicare l'offset per 4 occorre fare lo **shift left 2**, quindi shiftare di 2 bit a sinistra
 - shift left 1 → devo spostare verso sinistra di 1 posizione i bit che compongono il numero da shiftare e aggiungere uno zero alla prima posizione lasciata vuota a destra, togliendo il primo bit in testa a sinistra
 - se il nuovo MSB è diverso dal precedente c'è overflow
- l'operazione di shift left viene fatta 2 volte per far sì che avvenga la shift left 2
- lo spiazzamento calcolato in questo modo viene sommato al PC

- nell'ALU entra come operando B lo spiazzamento, quindi come ALUSrcB è settato ad 11
- invece come ALUSrcA è settato a 0 per selezionare il PC
- occorre dire all'ALU di effettuare un'operazione di somma quindi ALUOp viene settato a 00 per specificare un'operazione di **addizione** – viene calcolato così l'indirizzo di branch
- l'indirizzo di branch viene “parcheeggiato” in ALUOut ed eventualmente usato nella fase di EX nel caso in cui si tratti di un'istruzione di branch

ADD + TUTTE LE ISTRUZIONI R-TYPE – EXECUTE

3° COLPO DI CLOCK

- iniziamo la fase di execute con i registri A e B che contengono i dati delle locazioni del Register File
- nella fase di execute andiamo ad effettuare l'operazione di somma/ogni altra operazione aritmetico-logica tramite la ALU
- dobbiamo quindi dare in input alla ALU i contenuti dei registri A e B
- perciò ALUSrcA assumerà il valore 1 e ALUSrcB assumerà il valore 00 → selezioniamo i due operandi
- ALUOp serve a distinguere quali operazioni la ALU deve andare ad effettuare abbiamo 3 casistiche:
 - 1) quando vale 00 quindi sappiamo già che si tratta di un'operazione di somma, ALUControl non considera nemmeno il funct, ultimi 6 bit, e dice direttamente all'ALU di fare un'operazione di addizione
 - nella fase di fetch per calcolare $PC + 4$ e nella fase di decode per il calcolo dell'indirizzo di branch viene settato ALUOp a 00
 - 2) quando ALUOp vale 01, ALUControl saprà già che si tratta di un'operazione di sottrazione e quindi dirà all'ALU di eseguire direttamente un'operazione di sottrazione
 - per calcolare EPC ($PC - 4$)
 - per eseguire la branch on equal (beq)
 - 3) quando ALUOp vale 10, ALUControl si servirà anche degli ultimi 6 bit dell'istruzione per capire nello specifico quale istruzione aritmetico-logica deve essere eseguita

- ★ **ogni volta** che è necessario eseguire **un'addizione o sottrazione** nell'intero **ciclo di Fetch**, la ALU sarà pilotata con i bit 00 (somma) e 01 (sottrazione) → **non consideriamo** i bit 6 funct
 - es. nella Fetch → $PC + 4$
 - es. lw / sw per calcolare l'indirizzo della word in memoria da caricare / memorizzare
 - es. beq per confrontare due numeri ($x - y$)
- ★ **ECCEZIONE !!** → per le istruzioni **R-Type** nella fase di **Execute**, ALUOp assumerà il **valore 10** e l'operazione da far eseguire all'ALU sarà specificata prendendo in considerazione gli ultimi 6 bit dell'istruzione (**funct**)

4° COLPO DI CLOCK

- andiamo a scrivere nel registro destinazione, che si trova nel Register File (specificato in ogni istruzione R-Type), **il risultato della computazione** che troveremo in ALUOut
- dobbiamo scrivere il risultato contenuto in ALUOut nel registro del Register File quindi il segnale **MemtoReg** sarà impostato a **0**
- la scrittura nel Register File deve essere **abilitata**, quindi il segnale **RegWrite** deve essere asserito a **1**
- per capire in che registro scrivere il dato da salvare dobbiamo asserire **RegDst** a **1** che ci permette di considerare il registro destinazione indicato nell'istruzione

BRANCH ON EQUAL**3° COLPO DI CLOCK**

- In ALUOut abbiamo salvato l'indirizzo di branch su cui atterrare, calcolato nel colpo di clock precedente (Decode)
- in A e B fluiranno i dati contenuti nei registri del Register File → **ALUSrcA = 1 / ALUSrcB = 00**
- l'obiettivo è saltare al Branch Address se gli operandi sono uguali e per fare ciò si deve fare la sottrazione tra A e B quindi **ALUOp = 01**
- se la differenza è **zero** dall'ALU si "illuminerà" l'uscita chiamata "zero" che verrà asserita e invierà il segnale 1 ad una porta AND che come altro bit di entrata prende il segnale **PCWriteCond** il quale viene asserito ad **1** (per permettere di far fluire nel PC il Branch Address)
- **PCSource = 01** che fluisce nel mux per selezionare l'indirizzo di branch salvato in ALUOut
- se i numeri sono uguali, zero invierà **1** come segnale e **avverrà il salto**

JUMP**3° COLPO DI CLOCK**

- consideriamo i bit dell'istruzione "parcheggiata" nell'Instruction register che vanno dal [25 – 0] quindi gli ultimi 26 bit dell'istruzione
- successivamente vengono fatti fluire nello shift left 2, così da eseguire una moltiplicazione per 4 e da ottenere quindi 28 bit
- a questi 28 bit verranno aggiunti a sinistra (quindi dove vi sono i bit più significativi) i bit del PC (incrementato, che punta all'istruzione successiva) che vanno dal [31 – 28]
- ora avremo calcolato l'indirizzo di Jump e potremo selezionare l'opzione 2 del mux asserendo **PCSource = 10**
- in questo modo i bit dell'indirizzo di jump verranno fatti fluire nel PC

LOAD WORD**3° COLPO DI CLOCK**

- dall'Instruction Register fluirà nel Register file l'indirizzo del registro che contiene l'indirizzo base a cui dovremo sommare l'offset (ultimi 16 bit) per ottenere l'indirizzo in memoria che conterrà il dato da scrivere nel Register File
- fluiranno nella Control Unit i bit op che piloteranno il datapath e faranno fluire il segnale **ALUOp = 00** all'ALUControl per preparare l'ALU ad eseguire una somma
- gli operandi di questa somma saranno il contenuto del registro base letto dal register file, che verrà fatto fluire in A
- l'operando A sarà selezionato tramite **ALUSrcA = 1**
- i 16 bit dell'offset fluiranno in sign-extend per allinearli ai bit con cui opera l'ALU (32)
- **ALUSrcB = 10** perchè vorremo considerare come secondo operando i bit dell'offset esteso
- il risultato dell'addizione verrà "parcheggiato" in ALUOut

4° COLPO DI CLOCK

- il risultato dell'addizione salvato in ALUOut fluisce nel multiplexor più a sinistra che ci permette di selezionare ALUOut mettendo i bit di **lorD = 1** così da **leggere dalla memoria** l'indirizzo appena calcolato → **MemRead = 1**, **MemWrite = 0**
- il contenuto del registro in memoria viene a sua volta "parcheggiato" in **Memory Data Register**

5° COLPO DI CLOCK

- il contenuto di Memory Data Register viene scritto nel registro destinazione rt del Register File
- il registro in cui dobbiamo salvare il dato che abbiamo prelevato dalla memoria è specificato nei bit [20 – 16] dell'istruzione contenuta nell'Instruction register → RegDst = 0 così da indicare questo indirizzo come l'indirizzo del registro dove scrivere il dato
- MemtoReg = 1 così da selezionare MDR come dato da scrivere
- RegWrite = 1 per abilitare la scrittura

STORE WORD

- Prende un valore dal register file e lo carica nella RAM (memoria)

3° COLPO DI CLOCK → quasi uguale a quello della lw

- dall'Instruction Register fluirà nel Register file l'indirizzo del registro che contiene l'indirizzo base a cui dovremo sommare l'offset (ultimi 16 bit) per ottenere l'indirizzo in memoria dove salveremo il dato letto dal Register file
- fluiranno nella Control Unit i bit op che piloteranno il datapath e faranno fluire il segnale ALUOp = 00 all'ALUControl per preparare l'ALU ad eseguire una somma
- gli operandi di questa somma saranno il contenuto del registro base letto dal register file, che verrà fatto fluire in A
- l'operando A sarà selezionato tramite ALUSrcA = 1
- i 16 bit dell'offset fluiranno in sign-extend per allinearli ai bit con cui opera l'ALU (32)
- ALUSrcB = 10 perchè vorremo considerare come secondo operando i bit dell'offset esteso
- il risultato dell'addizione verrà "parcheggiato" in ALUOut
- ★ nell'operando B verrà "parcheggiato" il dato da scrivere in memoria, che viene letto dal Register file

4° COLPO DI CLOCK

- il dato contenuto in B verrà fatto fluire fino a Write data nella Memory
- ALUOut conterrà l'indirizzo della locazione di memoria dove salvare il dato → il mux riceverà il segnale lorD = 1 per selezionare ALUOut come indirizzo su cui scrivere
- i segnali entranti in memoria saranno MemWrite = 1 e MemRead = 0 → in questo modo viene scritto in memoria il dato contenuto in B

ESERCITAZIONE DATAPATH (3°-4°)

In riferimento allo schema completo del datapath multiciclo, in che fase (fetch, decode, execute), e come, sono impostati i segnali di controllo per le diverse classi di istruzioni?

- L'attivazione dei segnali di controllo per un datapath multiciclo può essere descritta da un automa a stati finiti
- I segnali di controllo (next_state per l'automa) sono definiti **in funzione dello stato corrente e dell'Opcode**
- **Per tutti i tipi di istruzioni**, nella fase di fetch (stato 0) sono impostati i seguenti segnali:
 - lorD=0 per selezionare PC come sorgente per l'indirizzo di memoria

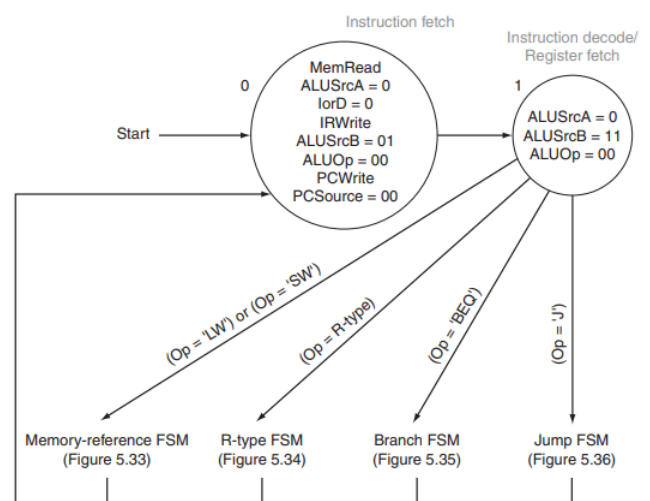
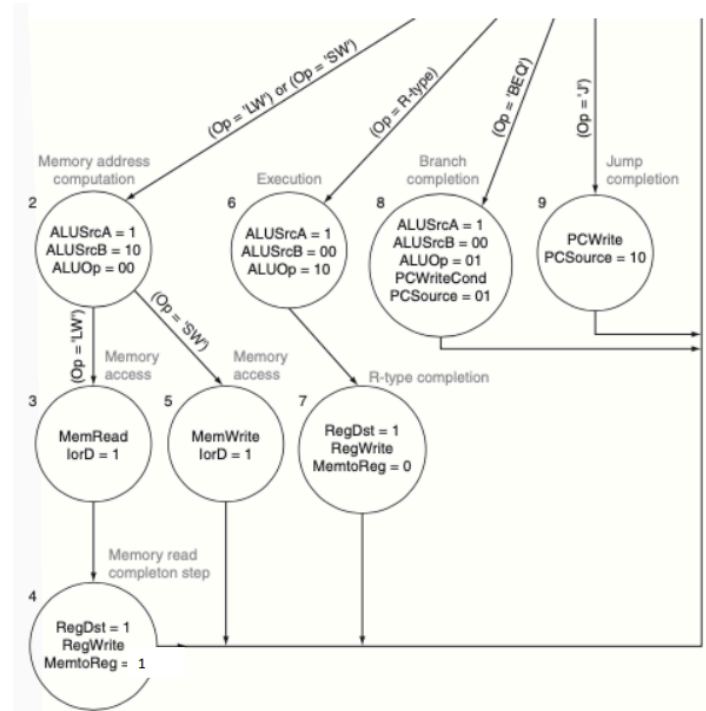


FIGURE 5.32 The instruction fetch and decode portion of every instruction is identical. These states correspond to the top box in the abstract finite state machine in Figure 5.31. In the first

- MemRead → per leggere un'istruzione da memoria
- IRWrite → per scrivere l'istruzione letta nell'Instruction register
- ALUSrcA, ALUSrcB, ALUOp, PCWrite, e PCSrc sono impostati per calcolare $PC + 4$ e memorizzare il nuovo valore nel PC
- **Per tutti i tipi di istruzioni**, nella fase di decode (stato 1) sono impostati i segnali ALUSrcA, ALUSrcB e ALUOp per calcolare il valore di branch
- Nella fase di execute sono impostati i segnali di controllo, **in funzione del tipo di istruzione in esecuzione** (paragrafo successivo)
- le istruzioni di jump o branch, dopo lo stato 1, richiedono un ciclo di clock per il loro completamento
- le istruzioni di tipo R-type e le istruzioni di memorizzazione (store), richiedono due cicli di clock per l'esecuzione di una sequenza di due stati d'impostazione dei segnali
- le istruzioni di lettura da memoria richiedono tre cicli di clock per l'esecuzione di una sequenza di tre stati d'impostazione dei segnali

Per quali classi di istruzioni, e come, è utilizzata la ALU (nella fase di execute)?

- La ALU (nella fase di execute) è utilizzata per
 - **istruzioni R-type** → per esecuzione dell'operazione indicata tra i due operandi
 - **istruzioni di accesso a memoria (lw/sw)** → per calcolare la somma tra l'offset e il contenuto del registro base
 - **istruzioni di salto condizionato (branch)** → per il confronto dei valori contenuti nei due registri che compaiono nella condizione. L'indirizzo di salto è già in ALUout perché è stato calcolato nella fase di decode
- La ALU non è usata nella fase execute di istruzioni jump



Aggiungere l'istruzione jr rs al set delle istruzioni del datapath multiciclo. Specificare le modifiche da effettuare al datapath e le modifiche da effettuare nella FSM relativa – diverse soluzioni possibili

- L'istruzione che si vuole implementare (v. pagina A-64 del libro di testo) determina l'aggiornamento del PC con i 32 bit contenuti nel registro il cui indirizzo è indicato dai 5 bit [25:21] dell'istruzione
 - Tenendo conto che per ogni istruzione, nella fase di decode, viene scritto
 - in A il contenuto del registro il cui indirizzo è indicato dai 5 bit [25:21] dell'istruzione
 - in B il contenuto del registro il cui indirizzo è indicato dai 5 bit [20:16] dell'istruzione
- Per realizzare l'effetto desiderato, potremo impostare i segnali di controllo in modo tale da scrivere in PC il risultato della somma tra il contenuto del registro A e il registro B:

Jump register

jr rs	0	rs	0	8
	6	5	15	6

Unconditionally jump to the instruction whose address is in register rs.

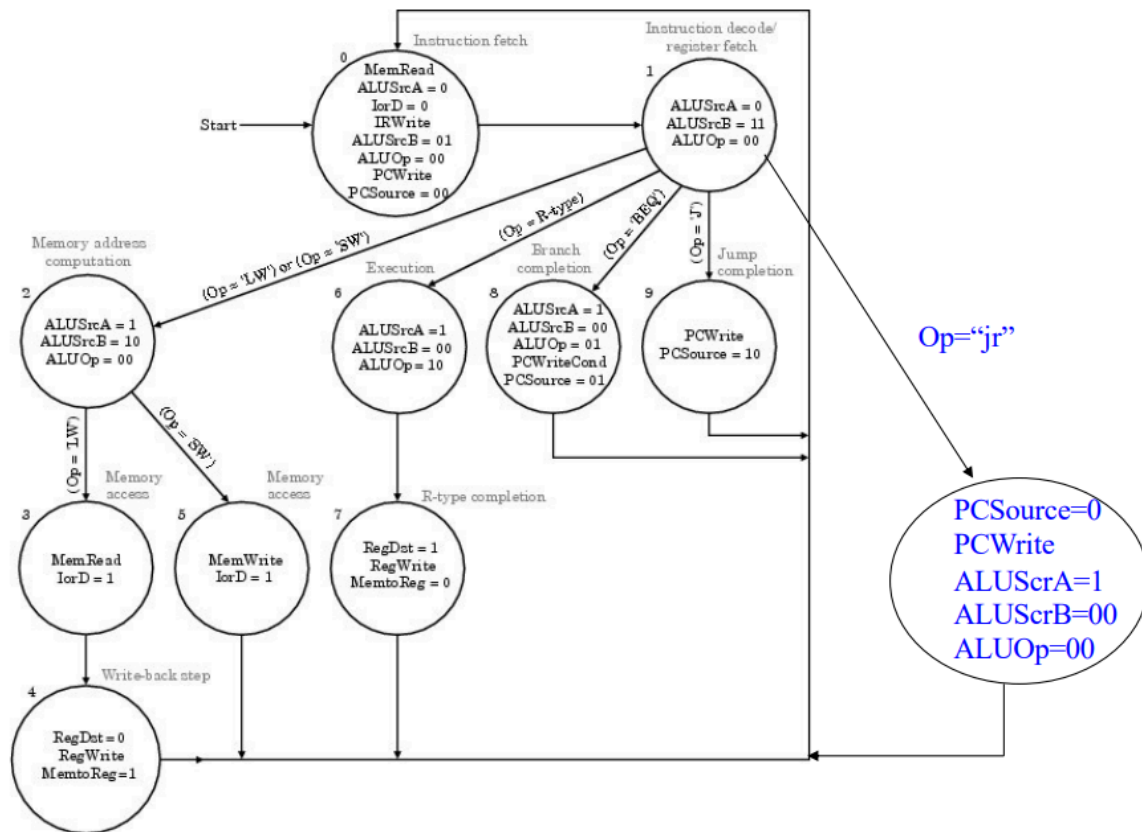
- abilitare la scrittura del PC (PCWrite=1)
- selezionare l'output della ALU come valore da scrivere nel PC (PCSource=0)
- selezionare il registro A come primo input per la ALU (ALUSrcA=1)
- selezionare il registro B come secondo input per la ALU (ALUSrcB=00)
- sommare (ALUOp=00)

PCSource=0

PCWrite ALUSrcA=1

ALUSrcB=00

ALUOp=00



LE ECCEZIONI

Durante l'esecuzione delle istruzioni si possono verificare degli eventi inattesi:

- 1) **eccezione, evento sincrono** generato all'interno del processore e provocato da problemi nell'esecuzione di un'istruzione
 - es. l'overflow (è un evento che accade in fase di execute ed è visibile solo con l'ALU – hardware – quindi bisogna modificare il datapath), istruzione non valida (bit di Opcode)
 - 2) **interruzione, evento asincrono** che giunge dall'esterno del processore: di solito arriva da un'unità di I/O (input e output devices, hardware) usata per comunicare alla CPU il verificarsi di certi eventi.
 - ★ interrupt → segnale che il processore riceve per interrompere il processo
 - es. la terminazione di un'operazione di I/O la cui esecuzione era stata richiesta dalla CPU
- sono delle variazioni rispetto alle *condizioni normali* di funzionamento della CPU
- Nei processori MIPS, una parte della CPU chiamata **coprocessore 0** registra le informazioni che il software necessita per gestire le eccezioni e le interruzioni.

ECCEZIONI	INTERRUZIONI
1) causate da eventi interni al processore	1) causate da eventi esterni al processore
2) sincrone rispetto al programma in esecuzione	2) asincrone rispetto al programma in esecuzione
3) la condizione di eccezione deve essere risolta da un gestore di eccezioni (handler di eccezioni)	3) sono gestite tra due istruzioni consecutive → sin termina l'esecuzione dell'istr. corrente, si gestisce l'interruzione, si riprende l'esecuzione con l'istr. successiva
4) se la condizione di eccezione è risolubile , allora il programma può riprendere l'esecuzione ; altrimenti il programma termina prima della sua <i>fine</i>	4) si sospende l'esecuzione del programma utente, si gestisce l'interruzione e poi si riprende l'esecuzione del programma utente

➤ **asincrone** → indipendenti dall'esecuzione del programma in corso (es. provocate dalle periferiche di I/O – input/output)

➤ **sincrone**:

- 1) richieste dal programmatore (es. syscall, break)
- 2) involontarie
 - elaborazioni con valori non ammissibili (**overflow** aritmetico generato dalla ALU durante la fase di **execute**)
 - **accesso ad indirizzi di memoria non corretti** (indirizzi di memoria non allineati o fuori dallo spazio del processo, generato dall'unità di controllo durante la fase di **decodifica**) → nella fase di Fetch il processore si rende conto che è stata letta una porzione di memoria non allineata al word, e tale eccezione viene gestita in fase di decode

In MIPS32 l'esecuzione passa ad una **locazione di memoria prefissata (0x80000180)**, dove si trova il gestore delle eccezioni (*exception handler*).

DEF.

L'**interrupt handler** è una parte di codice che è eseguito come conseguenza ad un'eccezione o interruzione. Questa routine (programma) si trova ad un indirizzo ben preciso della memoria:

LA GESTIONE DI ECCEZIONI E INTERRUZIONI

Il controllo del processore deve gestire gli eventi inattesi. Tutti i processori devono eseguire i seguenti passi per gestire un'eccezione/interruzione:

- 1) **interruzione dell'esecuzione del programma corrente** → interruzione dell'avanzamento predefinito del programma (ossia andare all'istruzione successiva)
 - l'**interrupt** è un segnale a livello HW che *blocca l'esecuzione del programma* a livello SW
 - il dato che viene sospeso, viene successivamente ripreso
- 2) **salvataggio parziale dello stato di esecuzione corrente** (istruzione corrente) in **hardware** (HW) – per *riprendere* eventualmente l'esecuzione del programma corrente
 - viene salvato il valore del **PC** dall'HW in eccezione all'interno del **registro EPC**
 - il **PC** viene **aggiornato** all'indirizzo che punta all'area di memoria in cui si trova l'exception handler

- 3) **salto ad una routine del sistema operativo (SO)** per gestire l'eccezione/interruzione (la routine è chiamata *handler o gestore delle eccezioni*, programma che gestisce le eccezioni a livello hardware) → livello hw perchè va ad interfacciarsi direttamente con i registri
- 4) esecuzione della routine del SO → l'**handler risolve** l'eccezione/interruzione
 - l'exception handler è collocato nello spazio **kernel text** a partire dalla locazione 0x80000180
- 5) **se possibile, ripristino dello stato di esecuzione** del programma e **continuazione dell'esecuzione**
 - ripristino dello stato al momento dell'eccezione (sarà svolta dal gestore dell'eccezione)
- i registri **K** sono riservati alla *gestione delle eccezioni* → **kernel mode**
 - Nei sistemi operativi che per la **protezione** fanno uso di meccanismi basati sul modello dei *domini gerarchici di protezione*, viene solitamente chiamato *kernel mode* (in inglese modalità del kernel) lo stato di **privilegio massimo** riservato all'esecuzione del **kernel**. Il codice in *linguaggio macchina* eseguito in tale modalità ha **accesso illimitato** alla **memoria, all'hardware** e alle altre risorse.
 - Un **kernel** in informatica costituisce il **nucleo** o cuore di un **sistema operativo**, ovvero il software che fornisce un **accesso sicuro e controllato dell'hardware** ai **processi** in esecuzione sul computer.
- il registro **\$at** viene usato dall'assembler → usato dal SW e dal gestore → prima che avvenga l'eccezione, il registro \$at contiene un *valore usato dall'assembler*. Quando avviene un'eccezione, siccome il valore di \$at **può essere alterato dall'handler**, il valore di **\$at viene salvato** per poterlo **recuperare** in fase di *ripristino del programma*

GESTIONE DELLE ECCEZIONI

Problema → come capire l'evento inatteso verificatosi?

Due possibili soluzioni:

- 1) **Indirizzo fisso** → in un **registro interno** deve essere salvato un **identificatore numerico** che identifica il **tipo di eccezione** verificatosi
 - l'handler *accede* a tale registro interno, salvato nel registro **Cause**, per determinare la **causa dell'eccezione**
 - vi è **un solo handler** per *tutti i tipi di eccezione*
 - 2) **Interruzioni vettorizzate** → esistono **handler diversi** per la gestione di *eccezioni/interruzioni differenti*
 - il *controllo della CPU* sceglie l'**handler corretto** → il controllo *salta all'indirizzo corretto*
 - per fare ciò viene predisposto un **vettore di indirizzi**, uno per ogni *tipo di eccezioni/interruzioni*
 - il **codice numerico** dell'eccezione/interruzione (Cause) permette di accedere alla **locazione di memoria corretta** in cui si trova l'handler che è in grado di gestire la specifica eccezione incontrata
 - tale *locazione di memoria* è contenuta nel vettore di indirizzi
- In MIPS viene adottata la **prima soluzione**, usando un registro chiamato **Cause** per memorizzare il *motivo dell'eccezione*.
- L'indirizzo dell'istruzione corrente che ha *causato l'eccezione* viene salvato nel registro **Exception Program Counter (EPC)**

Consideriamo solo **2 cause di eccezione**:

- 1) **Istruzione non valida** → generata dall'**unità di controllo** dopo la fase di decode dell'istruzione → nel momento in cui cerca l'istruzione da eseguire non la trova perchè viene

inserito un codice Op (Opcode) che non esiste → gli stiamo chiedendo di eseguire un'operazione che corrisponde ad un **codice inesistente**

➤ L'unità di controllo rileva tale eccezione sulla base dell'OPCODE di un'istruzione

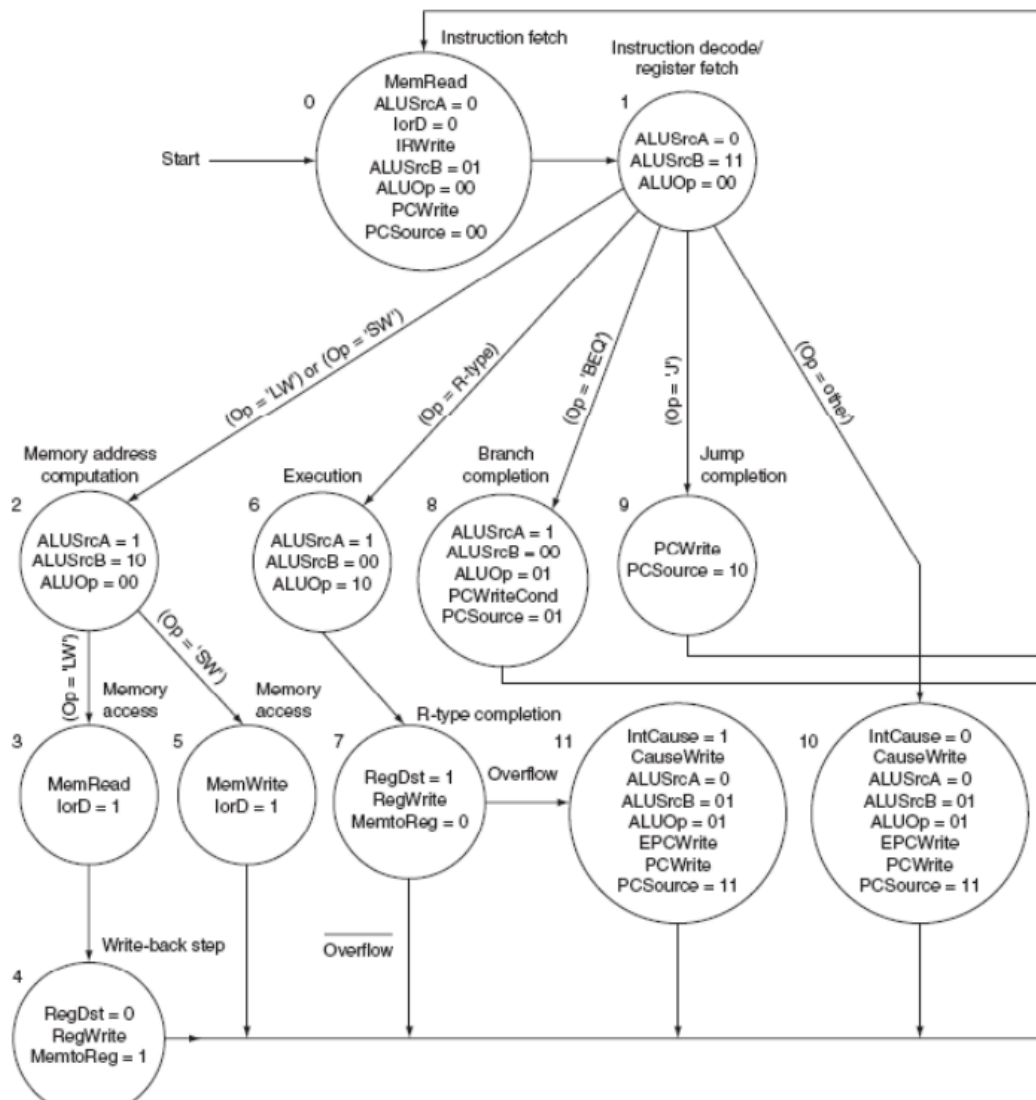
2) **Overflow** → generata dall'**ALU** nella fase di **execute** di un'istruzione **R-type**

➤ segnale che arriva dall'ALU all'unità di controllo

★ da aggiungere 2 nuovi stati alla FSM (Finite States Machine) del datapath

○ tali stati devono:

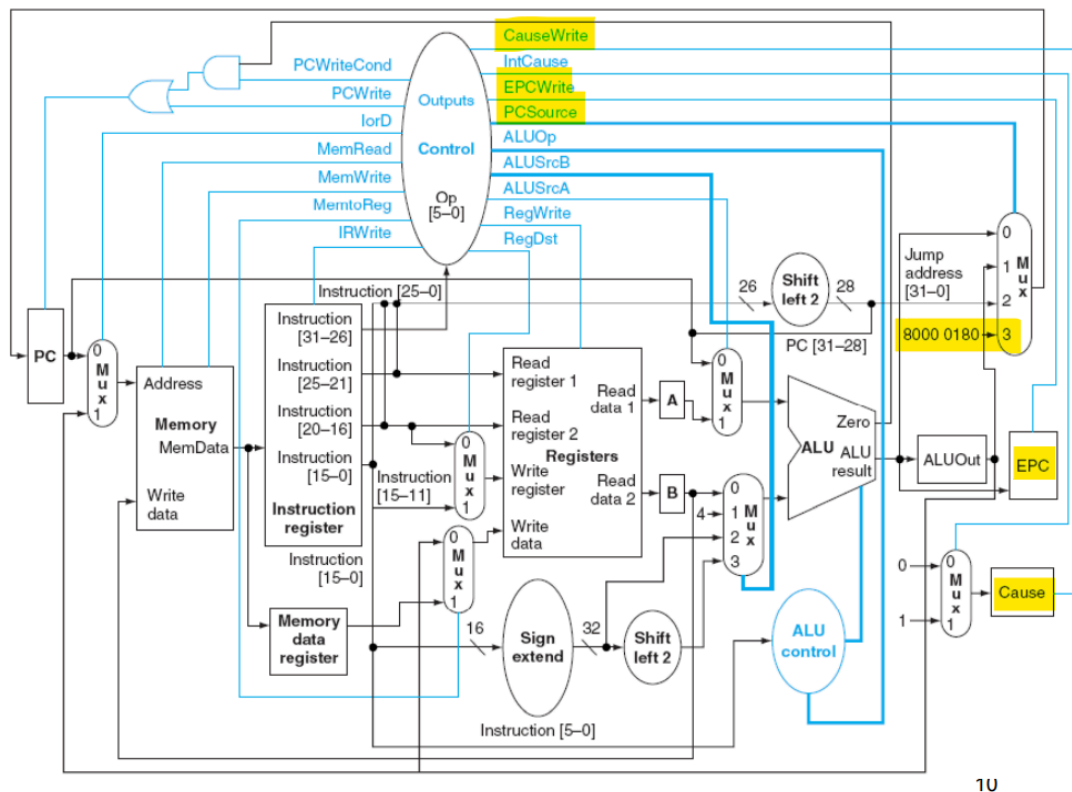
- salvare in EPC il valore $PC - 4$
- salvare nel registro Cause la causa dell'eccezione (0 o 1 in questo caso perchè consideriamo 2 esempi di eccezione)
- salvare in PC l'indirizzo del gestore delle eccezioni



Passi da seguire per la **gestione delle eccezioni**:

- 1) **individuare** l'evento inatteso, la causa dell'eccezione, e **salvare** quest'ultima in un registro dedicato denominato **Cause**
- 2) **interrompere** l'esecuzione corrente
- 3) **salvare** l'indirizzo dell'istruzione corrente in **EPC** ($EPC \leftarrow PC - 4$)
- 4) **saltare** a un gestore delle eccezioni del SO (sistema operativo) che si trova a un **indirizzo fisso** per gestire l'eccezione

DATAPATH MODIFICATO PER LA GESTIONE DI ECCEZIONI/INTERRUZIONI



10

Osservazioni:

- in MIPS non salva *nessun altro registro* oltre al PC
- è compito dell'handler (routine) salvare altre *porzioni dello stato corrente del programma*, se necessario
- esistono CPU dove questo *salvataggio* viene **prima** di saltare all'handler (approcci CISC)

EXCEPTION HANDLER

- **Porzione di codice assembly** che contiene il codice che gestisce le eccezioni
- Si trova alla **locazione fissa 0x8000180** in area **ktext** riservata al sistema operativo
- Dal momento che viene **eseguito dopo l'interruzione** di un programma utente *in un punto qualsiasi*, è tenuto a **preservare TUTTI i registri macchina**
- Ad esso sono riservati due registri **\$k0, \$k1** che può utilizzare *senza doverne preservare i contenuti*
- Al termine del trattamento dell'eccezione, l'exception handler *ritorna il controllo al programma interrotto* (istruzione **eret**)
- MIPS (e quindi anche SPIM) rileva e tratta le eccezioni **PRIMA** del completamento dell'esecuzione della istruzione corrente:
 - 1) se l'eccezione corrente era una **interruzione**, dopo la sua gestione, l'esecuzione del programma deve riprendere dall'istruzione corrente (salvata nel registro EPC)
 - 2) se si tratta di un **altro tipo di eccezione** ed è **possibile proseguire** (se non è avvenuto un errore irreparabile e non si tratta di una istruzione break), l'esecuzione deve riprendere dall'istruzione successiva (EPC+4)

ECCEZIONI SUL LIBRO

Il simulatore di MIPS32, SPIM, non implementa tutti i registri del *coprocessore 0*, i più importanti sono:

- **Status** → \$12 → interrupt mask and enable bits
- **Cause** → \$13 → exception type and pending interrupt bits
- **EPC** → \$14 → indirizzo dell'istruzione che ha causato l'eccezione
- si può accedere ai registri del coprocessore 0 tramite le istruzioni **mfc0** e **mtc0**
 - **mfc0** → move from coprocessor 0
 - **mtc0** → move to coprocessor 0
- dopo un'eccezione, il registro EPC **contiene l'indirizzo dell'istruzione in esecuzione** quando è avvenuta l'eccezione
- se l'eccezione è stata causata da un'**interruzione esterna**, allora l'istruzione non sarà stata eseguita
- tutte le altre eccezioni sono causate dall'esecuzione dell'istruzione in EPC
 - in ogni caso, un exception handler riprende il programma, ripartendo dall'istruzione salvata in EPC + 4
- se l'istruzione che ha causato l'eccezione ha effettuato un tentativo di accesso alla memoria ad un indirizzo **non allineato al word**, il registro BadVaddr contiene l'indirizzo della locazione di memoria non allineata, e quindi errata
- il campo dell'interrupt mask contiene un bit per ognuno dei 6 livelli hardware e 2 livelli software di interrupt
 - se il mask bit è 1, rende possibile a quel livello un'interruzione del processore
 - se il mask bit è 0 vengono disabilitati le interruzioni in quel livello
- le eccezioni e le interruzioni portano un processore MIPS a saltare ad una parte di codice all'indirizzo **0x80000180_{hex}** chiamato **exception handler**
 - questo codice esamina la causa dell'eccezione e salta ad un appropriato punto nel sistema operativo
 - il sistema operativo risponde ad un'eccezione o terminando il processo che ha causato l'eccezione, o compiendo alcune azioni
 - 1) un processo che causa un errore, come eseguire un'istruzione non implementata, viene terminato dal sistema operativo
 - 2) altre eccezioni sono invece delle richieste da un processo al sistema operativo di eseguire un servizio. Il sistema operativo processa la richiesta e ripristina il processo
 - 3) l'ultimo tipo di eccezioni sono le interruzioni da un dispositivo esterno → queste portano il sistema operativo a muovere dei dati in o da una periferica I/O e ripristina il processo interrotto
- l'exception handler non può sempre saltare all'istruzione che segue EPC
 - ad esempio, se l'istruzione che ha causato l'eccezione era in un'istruzione di branch, la prossima istruzione ad essere eseguita potrebbe non essere l'istruzione successiva in memoria

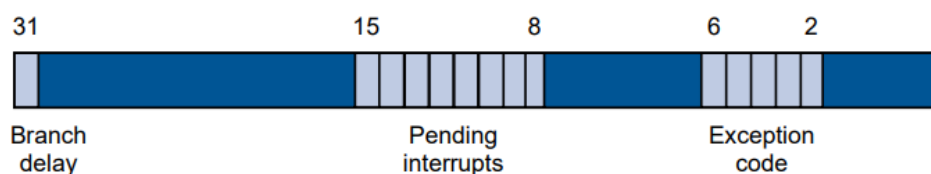


FIGURE A.7.2 The Cause register.

Number	Name	Cause of exception
0	Int	interrupt (hardware)
4	AdEL	address error exception (load or instruction fetch)
5	AdES	address error exception (store)
6	IBE	bus error on instruction fetch
7	DBE	bus error on data load or store
8	Sys	syscall exception
9	Bp	breakpoint exception
10	RI	reserved instruction exception
11	CpU	coprocessor unimplemented
12	Ov	arithmetic overflow exception
13	Tr	trap
15	FPE	floating point

DEFINIZIONE DATAPATH IN PIPELINE

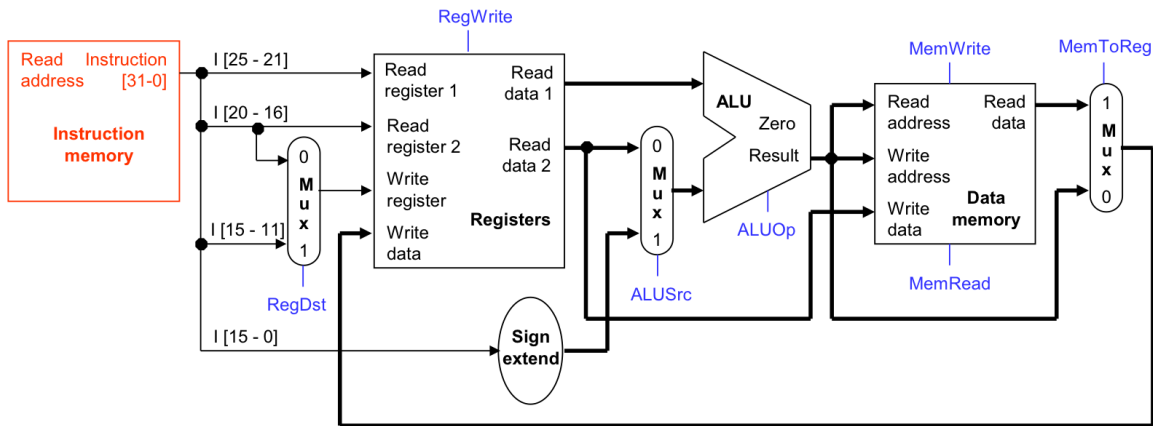
Quello che vedremo sono **Pipeline** da datapath a singolo ciclo.

Pipeline → **ottimizzazione** del *datapath* che accelera le computazioni.

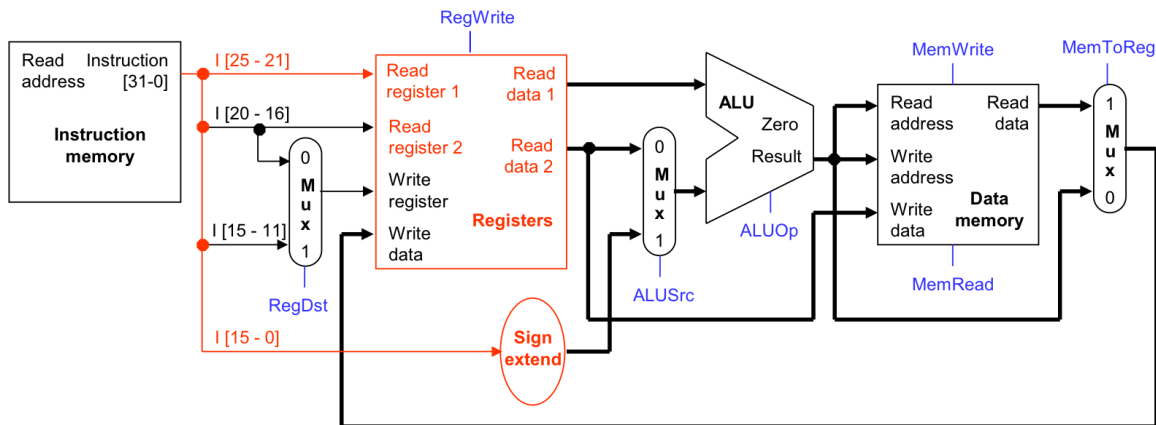
- sfrutta un **singolo colpo di clock** per utilizzare **diverse sezioni del datapath** in modo che possano essere **utilizzate contemporaneamente** su più istruzioni.
- Le **istruzioni**, come abbiamo visto, hanno **lunghezza variabile** rispetto alle operazioni che svolgono.
 - **IF** → *Instruction Fetch*
 - **ID** → *Instruction Decode*
 - **EX** → *Execute*, in cui si effettuano le computazioni per un'istruzione R-Type o per le Branch
 - **MEM** → *Memory*, in cui si leggono o scrivono dati in memoria
 - **WB** → *Writeback*, in cui si memorizza il risultato di una computazione nel registro destinazione
- Ciascuna di queste operazioni richiede **un ciclo di clock**.
- Queste operazioni, messe insieme in una quantità variabile, costruiscono l'**effettiva e completa esecuzione** di un'istruzione.
- Ogni "parte" di datapath, *in un singolo colpo di clock*, sarà dunque responsabile di uno dei 5 passaggi sopra citati e lo farà **contemporaneamente** alle altre "parti" ma su **istruzioni diverse**.

★ **RIPASSO DELL'ISTRUZIONE LW NEL DATAPATH SINGOLO CICLO**

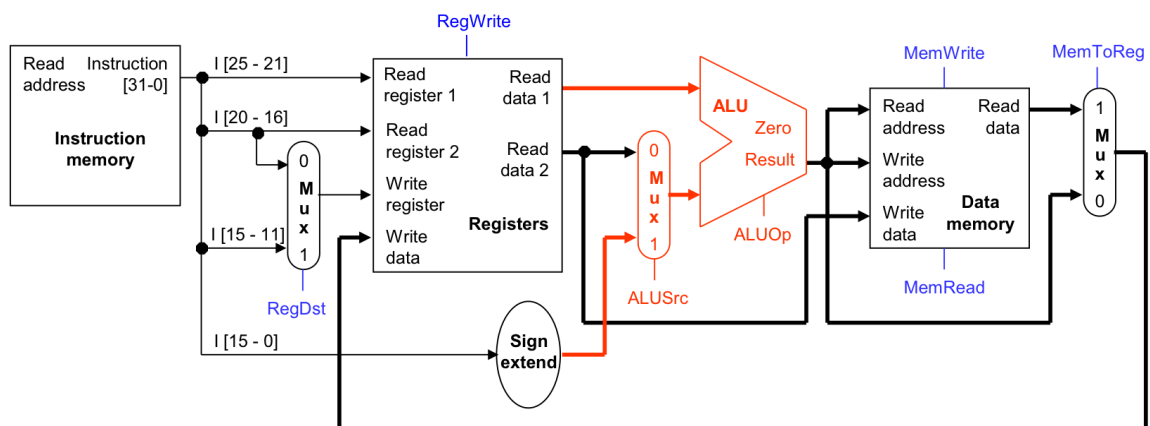
- Ignoriamo l'incremento del PC e il calcolo dell'indirizzo di branch
- 1) Nel passo per compiere l'**operazione di Fetch (IF, instruction Fetch)**, l'instruction memory viene letta



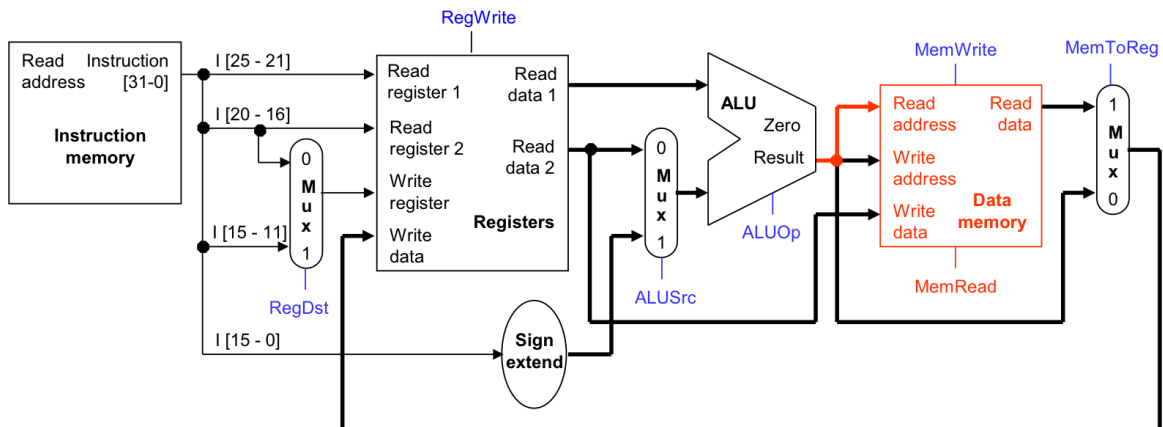
- 2) Nel passo in cui viene fatta l'**operazione di Decode (ID, instruction Decode)** viene letto il registro d'origine (source register), ossia l'informazione che si sta eseguendo, dal register file



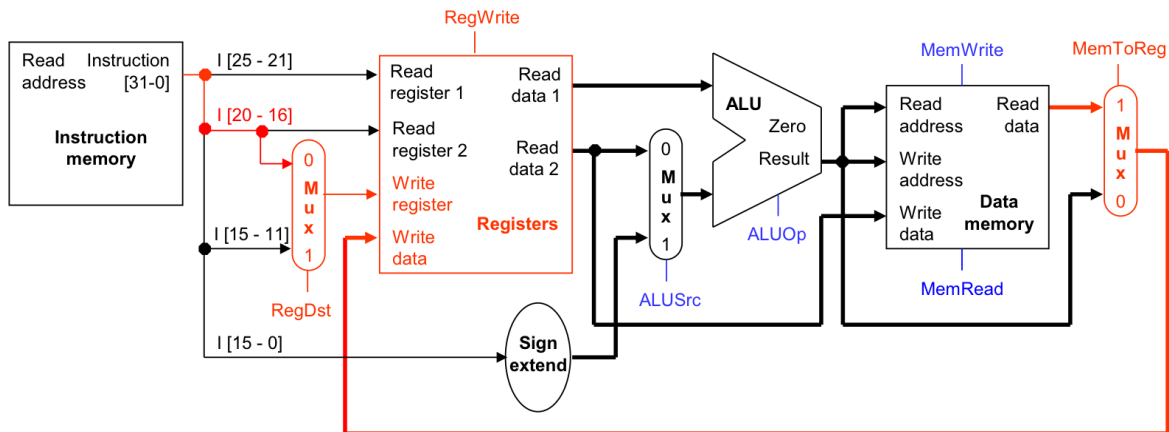
- 3) Nel passo di **Execute (EX)**, calcola l'effettivo indirizzo di memoria dal *source register* e dall'*immediate* (source register + immediate)



- 4) Il passo che riguarda la **Memoria (MEM)**, viene letto il dato in memoria dall'indirizzo calcolato dall'ALU

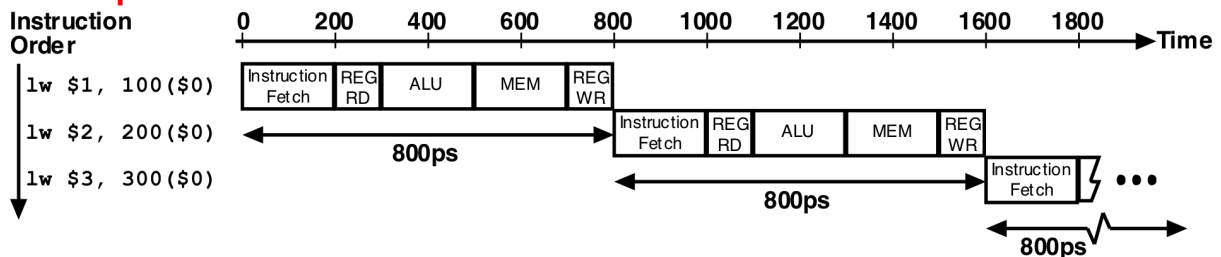


- 5) Infine nel passo di **WriteBack (WB)**, il valore nella memoria è salvato nel registro di destinazione



★ DATAPATH SINGOLO CICLO

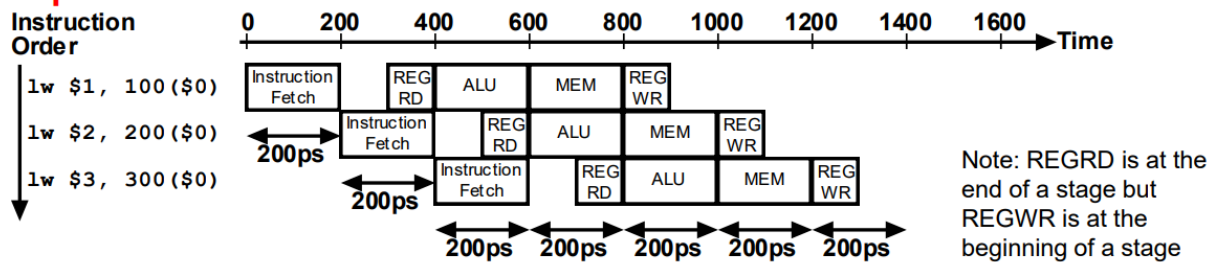
Non-Pipelined



★ DATAPATH CON PIPELINE

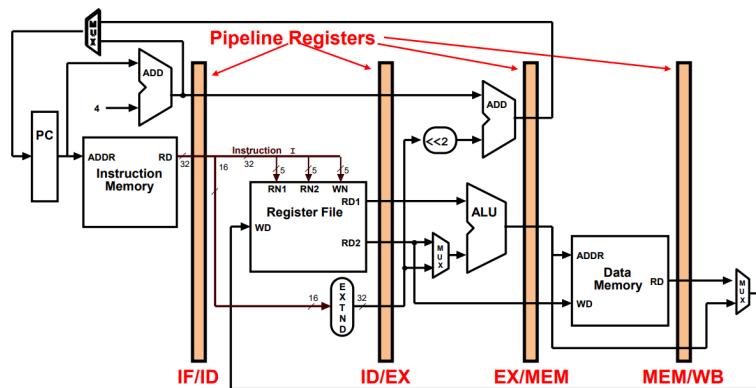
- Il **pipelined datapath** permette di *sfruttare continuamente ogni sua parte*, in quanto, quando **termina una delle 5 operazioni**, tale parte del datapath *non resta inutilizzata*, ma viene subito *impiegata* per compiere quella stessa fase per l'**istruzione successiva**
- in questo modo viene **risparmiato molto più tempo**, perchè non si deve aspettare il termine dell'esecuzione di un'intera istruzione, bensì solo di una delle sotto operazioni necessarie al compimento dell'intera istruzione

Pipelined



- L'idea di **pipeline** nasce dalla considerazione che nell'esecuzione di un'istruzione, lungo un colpo di clock, ci sono **sezioni "ferme"** per alcuni istanti di tempo e che potrebbero dunque essere invece *utilizzate*.
 - Le **istruzioni** vengono dunque **parallelizzate** nella loro **esecuzione**, risultando in una **capacità di trasmissione (throughput)** maggiore.
 - Per poter implementare questa idea, è necessario **aggiungere dei registri** che permettano, all'interno del singolo colpo di clock, di tenere **separate** le varie **sezioni** del datapath.
 - Questi **registri**, ad ogni colpo di clock, memorizzeranno il **valore in uscita** di ciascuna sezione, rendendolo **accessibile** alla sezione successiva.
 - Questa *separazione* permette di far **fluire il segnale solo** quando viene **aggiornato il registro**.
 - ★ Il risultato è che guardando il datapath in un certo istante vedremo **attive diverse unità**, *MA* su **istruzioni diverse**.
 - Dunque *2 istruzioni in sequenza* richiedono *più tempo* rispetto a *2 istruzioni eseguite in pipeline*.
 - A questo punto non si potrà più far *fluire un segnale di controllo* per **pilotare** un pezzo di datapath, perché bisognerà capire a **quale** delle n istruzioni entrate il segnale farà **riferimento**.
 - Dovremo dunque salvare (parcheggiare/appoggiare) ANCHE i **segnali di controllo** con dei **registri appositi**, così da **allinearli** ai quelli **uscenti da ciascuna sezione**.
 - In questo modo, il **segnale di controllo** che viene utilizzato in *una sezione datapath* **non si propagerà** anche *verso le altre sezioni*.
 - ★ Perciò i **nuovi registri** fotografano nel tempo sia i **segnali combinatori** che quelli di **controllo**.
- 1) Nell'eseguire una molteplicità di istruzioni, la **prima istruzione che entra** è quella **più vecchia**, che inizia con la fase di fetch, parcheggiando il **segnale del risultato** nel registro.
 - 2) Al **successivo colpo di clock** si farà fluire il segnale parcheggiato verso la **successiva sezione** del datapath che si occuperà della decode della **prima istruzione** e contemporaneamente si procederà alla fetch dell'**istruzione successiva**.
 - In pratica nel *secondo colpo di clock* il lavoro per la prima istruzione è nella **sezione ID**, mentre quello per la seconda istruzione è in **IF**.

Registri nella pipeline



- ★ E' importante notare che **non tutte le istruzioni** richiedono **tutte le operazioni**. Perciò dobbiamo implementare dei **collegamenti** che permettono ad alcune istruzioni di "scavalcare" sezioni del datapath.
 - Ad esempio, la **add** non ha bisogno di *memoria dati* ma del solo *write back* (dopo aver eseguito la somma). Perciò si dovrà introdurre un **collegamento diretto del segnale** che la EX ha generato verso la sezione WB che all'**ultimo colpo di clock** aggiornerà il registro con il *risultato*.

HAZARDS

- ★ **Hazard** → **situazione problematica** che può causare **malfunzionamenti** alla **pipeline**, in cui un'istruzione **non può seguire immediatamente** a un'altra.
- Ci sono **3 tipi** di hazard:
 - 1) **strutturale** → avviene quando **un settore di datapath** viene utilizzato contemporaneamente da **più istruzioni**.
 - 2) **di controllo** → avviene quando **la sequenza** in cui vengono **messe le istruzioni** può rendere sbagliate alcune decisioni. Si ha dunque quando si cerca di decidere prima che la condizione sia valutata.
 - 3) **di dati** → avviene quando **2 istruzioni** vogliono accedere agli stessi dati (registri), causando da parte di un'istruzione un *tentativo di accedere a un dato* che **non è ancora aggiornato**.

HAZARD STRUTTURALE

- Se avessimo **un'unica memoria**, potrebbe accadere che se in un dato istante *un'istruzione* vuole accedere alla memoria per una **load** e **contemporaneamente** *un'altra istruzione* vuole anch'essa accedere alla memoria per una **fetch**, avremmo un **problema**.
- ★ Questo tipo di hazard tuttavia è **già risolto**, perchè nella nostra implementazione a singolo ciclo abbiamo le **memorie separate**: una per i dati e una per le istruzioni.

HAZARD DI CONTROLLO

- Ad esempio può succedere che se avessimo una **add** seguita da una **branch** sui registri che dovrebbero essere stati **aggiornati** dalla **add**
- la branch decide se **saltare** o no sulla base di una **condizione**
- può succedere che la branch prelevi i dati che in memoria non sono ancora stati aggiornati → perchè non è ancora stata fatta l'operazione di WriteBack della **add** per l'aggiornamento dei dati
- in questo modo accade che la **branch** valuti la **condizione** rispetto a **dati non ancora aggiornati**.

HAZARD DI DATI

- Ad esempio 2 add in cui la seconda prende come operando il risultato della prima. Se fossero abbastanza vicine, si rischierebbe che la seconda effettui il calcolo sulla base di un **dato non ancora aggiornato**, perchè potrebbe succedere che al momento della fase EX della seconda, la prima debba ancora memorizzare il risultato nel registro destinazione.

SOLUZIONI AGLI HAZARD

Per tutti i tipi di hazard, la soluzione comune è lo **stall**.

- ★ **Stall** → consiste nel **congelare l'avanzamento delle istruzioni** che entrano in pipeline **per un certo numero di cicli di clock**.
- Tuttavia, con questa soluzione, si andrebbe contro all'*obiettivo* di avere **sfruttamento massimo** e **contemporaneo** del datapath e perciò le **prestazioni** ne risentirebbero.

1) STRUCTURAL HAZARD

- Per **structural hazard** su *accesso a memoria contemporaneo* la soluzione, come abbiamo già visto, consiste nel **separare la memoria**.

2) DATA HAZARD

1. Stall

- Nel caso di un **data hazard**, utilizzando la soluzione dello **stall**, l'avanzamento delle istruzioni che entrano nel datapath viene quindi **congelato**, finché il **dato** richiesto non è effettivamente **pronto** e quindi **aggiornato**.
 - Dunque, ad esempio, se ho **2 add** consecutive che lavorano sugli stessi registri, si decide di far aspettare ad entrare la seconda.
- Questo fenomeno dipende dunque dalla **distanza tra un'istruzione e l'altra**, che, se *troppo piccola*, causa appunto un **data hazard**.

2.

- *Separazione* della fase di aggiornamento e lettura all'interno di un *singolo colpo di clock*.
- In questo modo, se avessi che in un certo istante *un'istruzione voglia aggiornare un dato e un'altra leggerlo*, potrei effettuare l'*operazione di aggiornamento* sul fronte di salita del clock, così che nel fronte di discesa avvenga la *corretta lettura del dato aggiornato*.
- Tuttavia questo funziona **solo per certe distanze tra le istruzioni**. Per altre distanze invece si avrebbe comunque **hazard**.

3. Forwarding

- Comporta un **ulteriore cambiamento fisico del circuito**.
- Predisponiamo, con dei **collegamenti** nel datapath, la disponibilità diretta di un **segnale** a fasi successive.
- Consiste perciò nell'**inoltrare** (portare indietro) il **risultato** dell'operazione uscente dalla **ALU** verso un **operando** non appena questo è pronto, ovvero **senza aspettare la WB**.
- Da notare che *ciò che viene utilizzato per l'operazione successiva* è il **risultato** e **non il registro aggiornato**, che verrà invece *aggiornato* effettivamente *in seguito*.
- Chiaramente il risultato viene riportato indietro solo al **successivo colpo di clock**.

3) CONTROL HAZARD

1. Stall

→ Nel caso di **branch** l'attesa (**stall**) da introdurre sarà **maggiore**, perché deve essere *abbastanza grande* da far **aggiornare il PC** dopo che è stata verificata la condizione.

2.

→ **Modifica della struttura hw** della *pipeline*, per fare in modo che i **dati** con cui si *effettua la decisione* siano **aggiornati velocemente**.

3. Predict

→ **Predict** → "tirare a indovinare".

→ Nel caso di branch, consiste nel **prevedere l'esito del controllo**, ad esempio *considerando che il controllo fallisca sempre* e quindi prendendo sempre l'istruzione seguente alla branch piuttosto che quella a cui saltare nel caso in cui la condizione sia *verificata*.

→ Nel caso in cui la **predizione sia sbagliata**, si devono **eliminare** le sequenze di istruzioni accettate in pipeline, bisogna dunque "*sciacquarle via*" così da riportare i registri ai **valori intatti** *prima* della *branch*.

→ Questo **svuotamento** può essere **automatico** o **meno**.

4. Delay decision

→ Compilatore che genera una **porzione di codice** in *assembly* che, nell'ottica di *evitare* il problema, **sposta le istruzioni** adeguatamente per **evitare** il rischio di **hazard**.

INPUT / OUTPUT

Tecniche di *gestione input / output* (ingresso / uscita):

- periferiche input–output
- bus di sistema
- periferiche mappate in memoria
- controllo di programma
- banda passante e latenza
- input–output gestito direttamente da programma
- input–output con interruzioni
- input–output con DMA (**Direct Memory Access**)
- periferiche in SPIM

INPUT–OUTPUT

- **Input–Output** → insieme di architetture e dispositivi per il *trasferimento* di informazioni da e verso l'elaboratore
- dispositivi eterogenei per:
 - velocità di trasferimento
 - latenze
 - sincronizzazione
 - modalità di interazione (sia con l'uomo sia con la macchina)

BUS DI SISTEMA

Esistono *vari tipi di bus* nei computer di oggi.

- il **bus di sistema** è un *bus comune* che *collega* la **CPU** con la **memoria** e le **periferiche**
- il bus di sistema è composto da:
 - 1) **bus di dati** → sono le linee per *trasferire dati e istruzioni* da e verso *dispositivi*
 - 2) **bus di controllo** → *trasporta informazioni* per la definizione delle operazioni da compiere e per la sincronizzazione tra i *dispositivi*
 - 3) **bus degli indirizzi** → la **CPU** trasmette gli indirizzi di memoria o di periferica che *identificano i dati da leggere o scrivere dalla memoria o dalle periferiche*
- **tutte le unità** dell'elaboratore sono connesse al bus
- *vantaggi* → *elevata flessibilità, semplicità, basso costo*
- *svantaggi* → *gestione complessa del canale condiviso*

PERIFERICHE

- Le **periferiche** sono *dispositivi* per l'input-output *collegati* alla **CPU** tramite il **bus di sistema** e/o **interfacce**
- le **interfacce** sono standardizzate per la comunicazione
 - hanno una componente hardware (es. il controllore della periferica) e una componente software (es. il driver)
 - **firmware** → è il *programma* che *risiede* sulla **periferica** e le permette di eseguire le *operazioni di lettura/scrittura*
 - **driver** → il **SO** lo usa per *comunicare con la periferica*
- *struttura* di un'interfaccia (es. tramite i registri della periferica):
 - 1) contiene registri di dati (oppure **buffer**)
 - 2) ha associato registri di stato

PERIFERICHE MAPPATE IN MEMORIA

- una *parte della memoria* riservata al sistema viene usata per la comunicazione con le periferiche
- ogni periferica ha uno **spazio dedicato nella memoria** → a ogni *periferica* viene *assegnato* un **identificatore unico** (cioè indirizzo unico)
- i registri (dell'*interfaccia della periferica*) sono **mappati in memoria**
- tali registri non sono accessibili da un *programma utente*
- i *programmi utente* devono *passare dal SO* per accedere a questi registri riservati
- l'accesso alla periferica è *simile* all'*accesso alla memoria* (es. **lw**, **sw**) dal punto di vista della **CPU**
- osservazione → *non tutte le architetture* degli elaboratori usano questa soluzione
- sullo stesso bus in cui c'è la **RAM** ci sono i **registri della periferica**
- alcuni indirizzi ci fanno vedere *indirizzi di RAM* altri *indirizzi di periferica*
- ★ ogni periferica ha un suo spazio dedicato in memoria
- ★ in questo spazio risiedono dei **registri della periferica** che hanno una corrispondenza con i **registri** che risiedono in **memoria**
- ★ dal punto di vista della CPU *l'input/output con le periferiche* corrisponde ad un'azione di lettura / scrittura sulla **memoria** (sui *registri delle periferiche*, che sono *mappati in memoria*)

REGISTRI DI INTERFACCIA DELLA PERIFERICA

- 1) **registro di stato** della periferica
 - rappresenta lo **stato** (*busy / ready*) della periferica
 - viene letto dalla CPU
 - esempi di stato → periferica pronta per ricevere dati, dati richiesti disponibili per il trasferimento
 - la CPU interroga la periferica e le chiede se è pronta per il trasferimento dati → la periferica restituisce il suo stato
 - è la periferica a scrivere il suo stato, dato che è più lenta della CPU
- 2) **registro dei dati** della periferica
 - rappresenta i dati di input o di output in base al *tipo* della periferica
- 3) **registro di comando** (*controllo*) → dare un *ordine* alla periferica

PASSI DI I/O (versione molto semplificata)

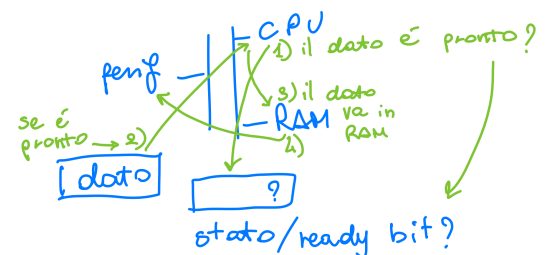
- 1) la CPU interroga lo stato della periferica
- 2) la periferica restituisce il suo stato
- 3) se la periferica è pronta a trasmettere o ricevere dati, la CPU richiede il trasferimento dei dati
- 4) la CPU invia o riceve i dati

TECNICHE DI GESTIONE I/O (“controllo di...”)

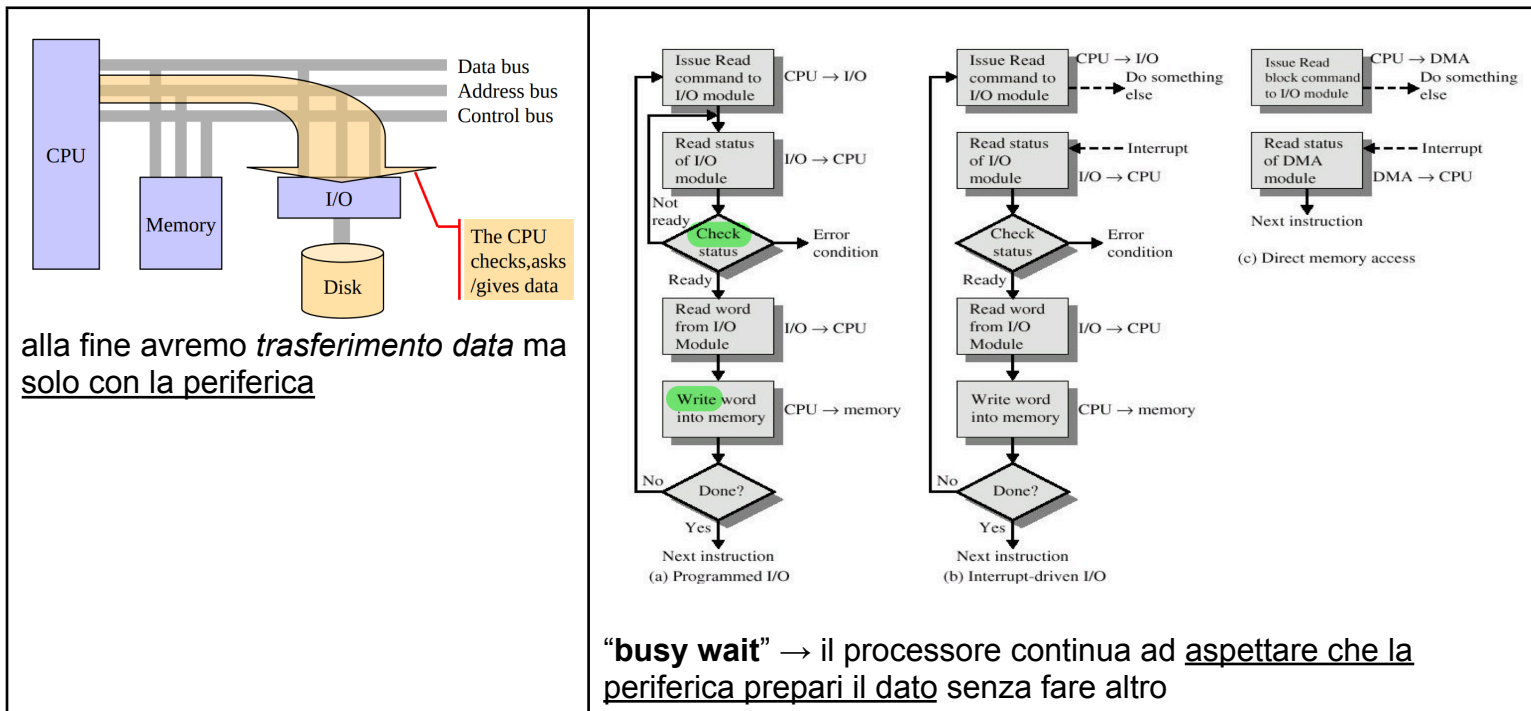
I/O gestito da programma (Programmed I/O)	Trasferimento via CPU	Utilizzo di interruzioni (uso di HW specifico)
I/O guidato da interrupt (Interrupt Driven I/O)		
Accesso Diretto alla Memoria (Direct Memory Access – DMA)		

I/O GESTITO DA PROGRAMMA

- 1ª modalità tutta SW
 - può avere *colli di bottiglia* → non sarà prestante
 - I/O gestito dal controllo di programma
 - la **periferica** ha un ruolo passivo
 - la **CPU** si occupa sia del controllo sia del trasferimento dati
 - la **CPU** predispone il controllore della periferica all'esecuzione dell'I/O
 - la CPU *si ferma* e **interroga** il registro di stato della periferica in *attesa che sia pronta* (es. **ready bit** assume un determinato valore)
 - vantaggio → *risposta veloce* al **ready bit**
 - svantaggio → la CPU bloccata in stato di **busy waiting**
- La gestione dei dispositivi I/O è totalmente affidata alla CPU.
- Si scansionano periodicamente tutti i dispositivi per verificare se qualcuno di essi richiede un servizio.

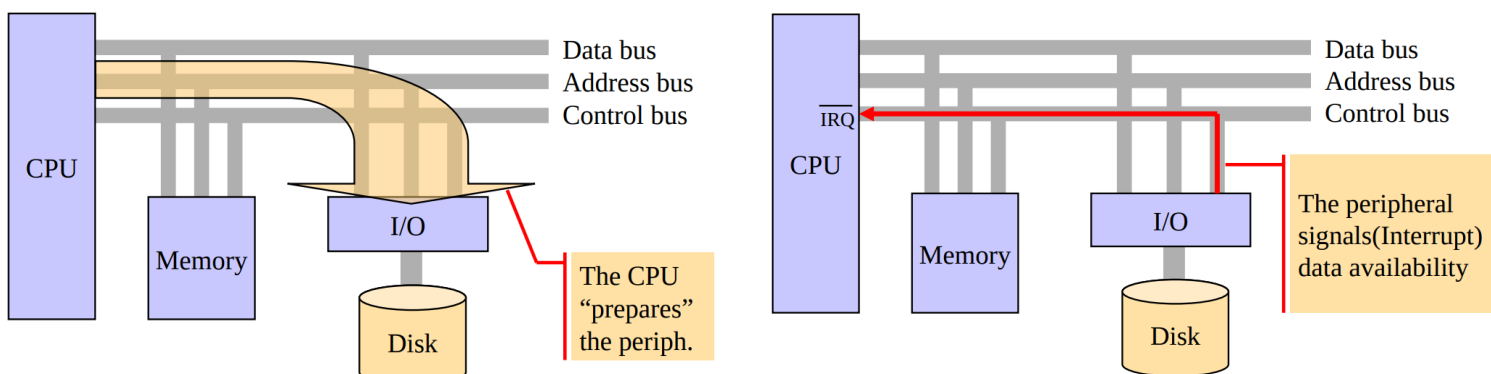


- La maggior parte del tempo la CPU è impiegata dal **programma principale** nell'esecuzione di un ciclo di **POLLING**.
- Viene normalmente utilizzato nei sistemi più piccoli e meno complessi, in quanto è *poco costoso* in termini di *hardware*, *poco efficiente*.

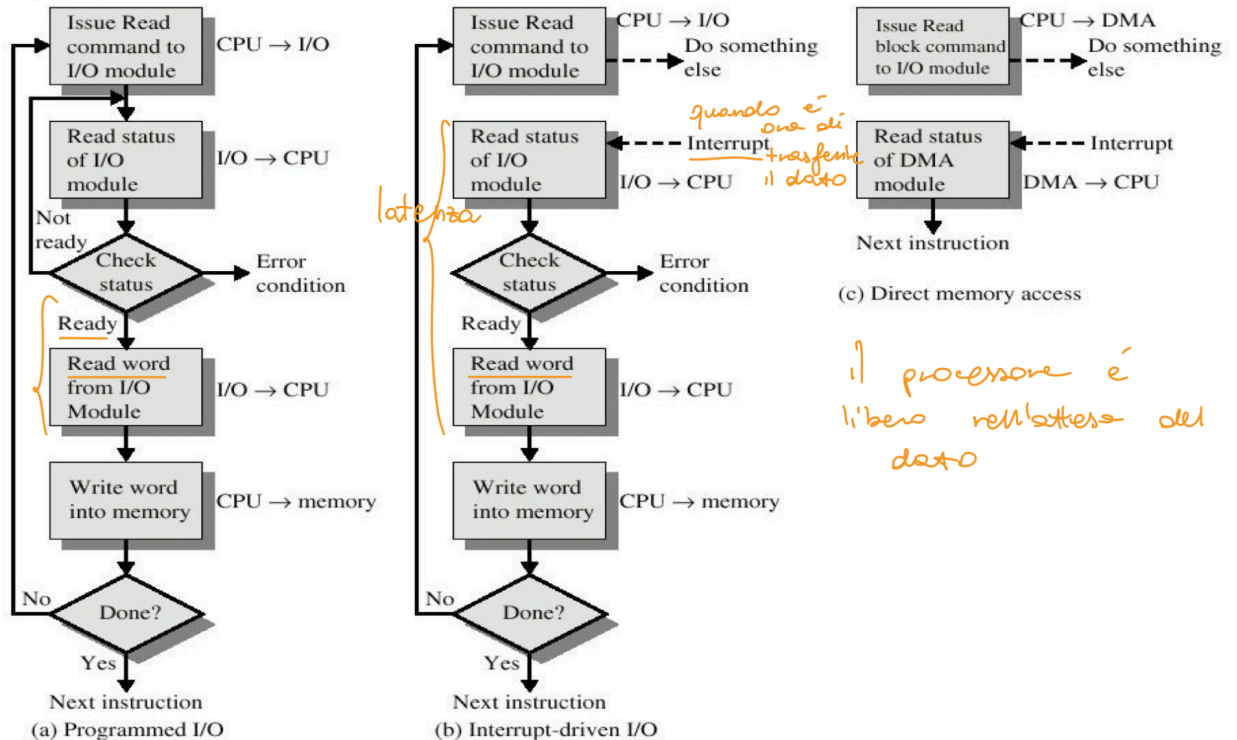


I/O GUIDATO DA INTERRUPT

- **Interrupt** → **evento asincrono** che genera l'interruzione del normale funzionamento del processore
- ➔ **Procedura:**
 - 1) la **periferica** segnala alla **CPU** di *aver bisogno di attenzione* mediante un segnale sul bus di controllo
 - 2) la periferica avvisa la CPU attivando un **segnale interrupt request**
 - 3) quando il **processore** se ne accorge (in una **fase di fetch**) *informa la periferica* con un **segnale interrupt acknowledge**
 - 4) la **CPU** interrompe l'esecuzione del programma corrente (*salvando il contesto dell'esecuzione del programma per poter riprendere la sua esecuzione*) ed esegue la procedura di risposta dell'interrupt
 - 5) terminata l'esecuzione della **procedura di interrupt**, la CPU *riprende l'esecuzione del programma interrotto*
 - 6) il **programma utente** *continua la sua esecuzione*



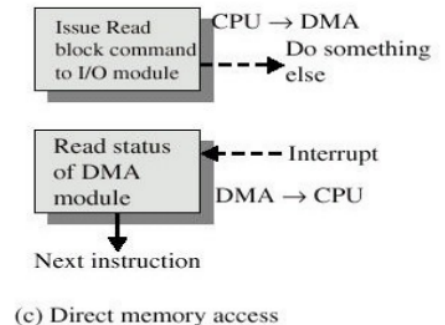
modo 2) risparmio risorse processore, introduco latenza



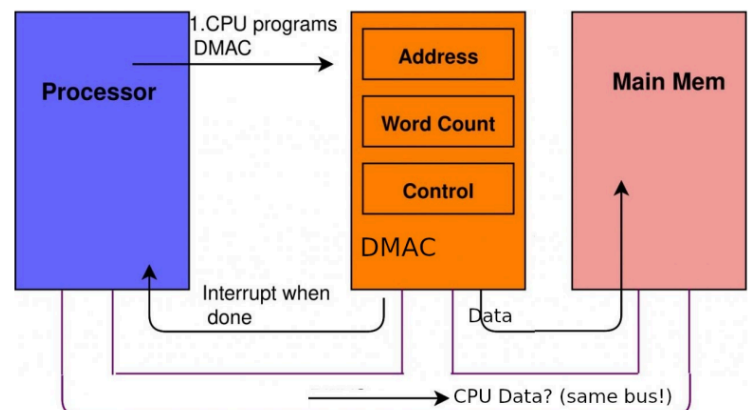
- vantaggio → la **CPU non** fa più **busy waiting** come per I/O gestito da programma
- salvataggio → la **CPU** deve comunque gestire le operazioni di trasferimento

DMA – ACCESSO DIRETTO ALLA MEMORIA

- per evitare l'intervento della CPU nella fase di trasferimento dati è stato introdotto il **protocollo di trasferimento DMA – Direct Memory Access**
- è un HW più sofisticato
- il **DMA** è usato quando si trasferiscono velocemente grandi quantità di dati
- con DMA la **periferica diventa autonoma nell'accesso alla memoria**
- è la periferica che gestisce i trasferimenti → la **CPU non interviene** nel trasferimento dei dati
- necessita **2 registri in più** per ogni periferica oltre altri **registri di stato e registro dei dati**
 - un registro che indichi l'**indirizzo di memoria da/dove trasferire i dati**
 - un registro che indichi la **quantità dei dati da trasferire**
- anche questi 2 registri aggiuntivi sono mappati in memoria
- alla fine del trasferimento la periferica invia un **interrupt** alla **CPU** per segnalare il completamento del trasferimento
- moduli aggiuntivi (hardware) sul **bus**



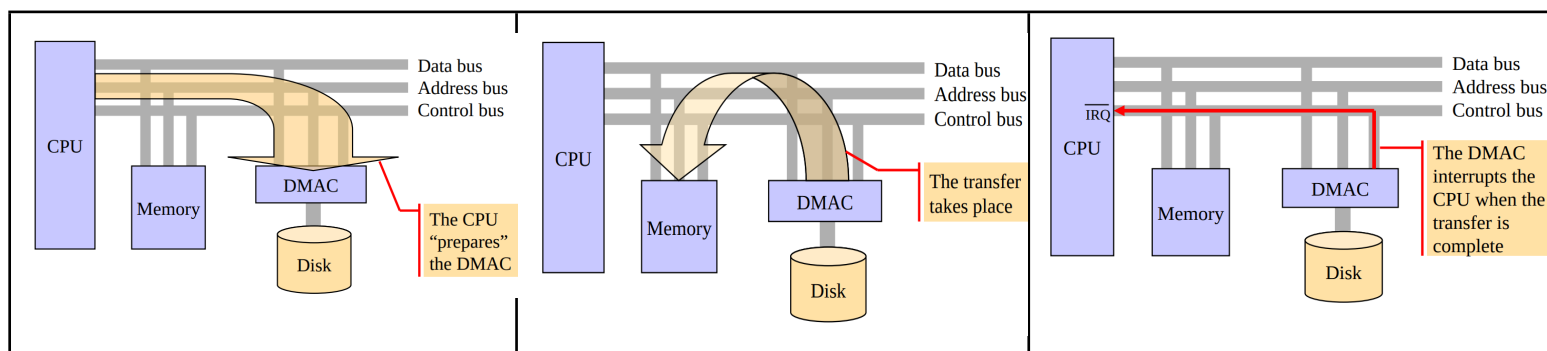
il processore è libero nell'attesa del trasferimento del dato



- il **DMA controller** occupa il **bus (RAM)** tra *CPU e memoria* per il trasferimento dei dati, quando il **bus** tra CPU e memoria **non è usato**
 - aspetta un momento in cui la **CPU non comunica con la memoria**, quindi si attende che il processore non utilizzi il bus, perchè la *priorità la ha la CPU*
 - **cycle stealing** → appropriamento del bus della CPU (più comune)

DMA OPERATION

- la CPU comunica al **DMA controller**:
 - se si tratta di un'operazione di lettura o scrittura
 - l'indirizzo del dispositivo dal quale si devono trasferire i dati
 - il primo indirizzo del blocco di memoria da trasferire
 - la quantità di dati che *deve essere trasferita*
- ★ il **DMA** esegue il trasferimento mentre la **CPU** è occupata ad eseguire altri processi
- ★ il **DMA** invia un **interrupt** quando è stato completato



LATENZA VS. BANDA PASSANTE

- La **latenza** è definita come il tempo per completare una specifica operazione
- La **banda passante** (chiamata anche **throughput**) è il numero di unità di lavoro (di un'operazione) che *possono essere completate* in una specificata unità di tempo
- le due misure sono molto diverse
- **banda passante** → *minor banda passante* in quanto il trasferimento di ogni dato necessita più tempo
- **latenza** → *maggiore* per la maggiore quantità di operazioni da eseguire
- se la banda passante è per esempio di 1KB, quindi *può trasferire 1000 byte* alla volta, ma l'utente sceglie di fare *tanti accessi veloci* per reperire *1 byte alla volta*, quella banda passante sarà *sfruttata solo per l'1%*, e questo non è efficiente per il sistema nel suo complesso
- è più efficiente reperire **blocchi contigui di memoria** per andare a sfruttare al *massimo* la banda passante
- **Per esempio**, un disco impiega generalmente 10ms per eseguire l'accesso, la maggior parte di questo tempo è impiegato per spostare la testa nel posto giusto (*seek time*), e aspettare per la rotazione del disco (*rotation delay*).
- se è richiesta una *piccola quantità di dati*, la minor latenza è ottenuta se solo quel pezzo di dati è letto dal disco
- se sono richiesti *molti piccoli pezzi del dato* che a livello di memoria si trovano in locazioni adiacenti, è più efficiente reperire l'intero blocco dati piuttosto che reperirne uno ad uno

- in questo modo verrà fatto **un solo accesso alla memoria**, si muove solo una volta la testa del disco e si impiegano solo 10ms per eseguire l'accesso in quanto viene fatto una sola volta (ipotesi), certamente si impiegherà *più tempo nel reperire l'intero blocco dati* in quanto si tratta di una grande quantità, ma questa operazione risulterà comunque *minore rispetto a reperire un dato alla volta*
- un'unica transazione di un **blocco contiguo di memoria** è più lenta (dato che il tempo per trasferire una più grande quantità di dati è maggiore), però l'effetto complessivo è un uso più efficiente della banda passante

PRESTAZIONI – BANDA PASSANTE E LATENZA

- *misure* che permettono di **valutare l'efficienza** della gestione delle operazioni di I/O
- **banda passante** → rappresenta la quantità di dati che *si può trasferire* per unità di tempo
 - rappresenta una misura di flusso
- **latenza** → rappresenta il **tempo** che *intercorre* tra l'istante in cui **una periferica è pronta** per il **trasferimento** e l'istante in cui **il dato viene trasferito**
 - è una misura di tempo

PRESTAZIONI – I/O GESTITO DA PROGRAMMA – POLLING

- **banda passante** → più alta possibile perchè la *CPU trasferirà subito il dato* (es. **molti dati** per un'unità di tempo) e la *gestione della periferica* richiede poche istruzioni
 - banda passante molto alta, in quanto vengono trasferiti i dati molto velocemente.
- **latenza** → minima in quanto la *CPU noterà subito lo stato della periferica* (tempo massimo: un *ciclo intero* di **busy waiting**)
 - latenza molto bassa in quanto il processore vede subito che la periferica è "ready"
- Esempio → se trasferisco più dati (1KB) si ha un ciclo interno per il trasferimento di ogni byte e un ciclo esterno per il trasferimento di tutti i byte

PRESTAZIONI – I/O GESTITO DA INTERRUPT

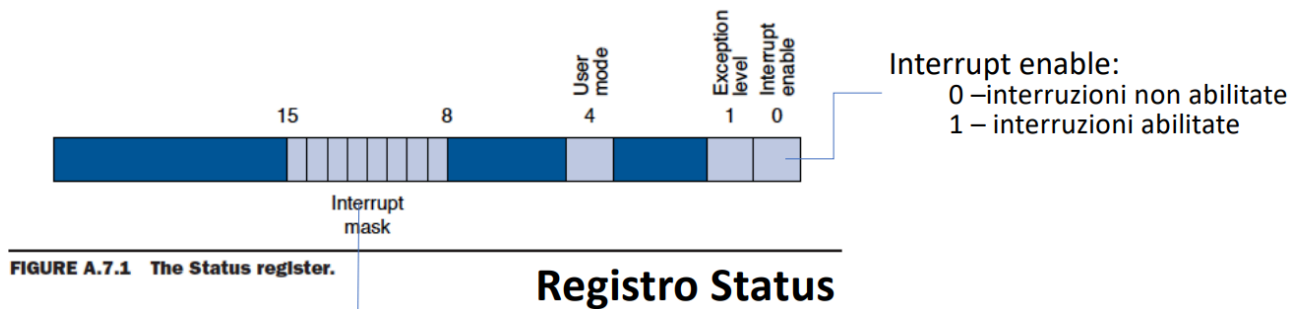
- **banda passante** → minor banda passante (*meno colli di bottiglia*) in quanto il *trasferimento di ogni dato* necessita di più tempo
 - banda passante bassa, perché i dati ci impiegano di più ad essere trasferiti rispetto al polling
- **latenza** → maggior latenza per la maggior quantità di operazioni da eseguire
 - latenza più alta rispetto al POLLING in quanto la cpu finisce prima di eseguire la sua istruzione e poi controlla se la periferica è ready

PRESTAZIONI – DMA

- **banda passante** → massima perchè la *CPU non deve eseguire nessuna istruzione*
 - banda passante massima, in quanto la cpu non esegue nessuna istruzione
- **latenza** → minima dato che nessuna istruzione è eseguita dalla *CPU*
 - latenza minima, perché nessuna istruzione viene eseguita dalla cpu, si "spreca" tempo solo per accedere al bus
- nel **DMA** il dato si muove in modo più diretto senza che sia *controllato / frenato dalla CPU*

ESERCITAZIONE I/O (3^a-5^a)**→ INTERRUPT**

- Sono tipicamente **causati da dispositivi hardware esterni** (es. *periferiche I/O*), collegati al **processore** attraverso delle **linee di controllo di bus**
- Quando le **periferiche** sono **pronte ad effettuare operazioni** impostano una **linea di interruzione** collegata alla **CPU**
- Quando viene **attivata una linea di interruzione**, se il sistema ha **abilitato le interruzioni della CPU**, il **normale flusso di esecuzione** viene **interrotto**, e la **CPU** passa a **gestire l'interrupt**

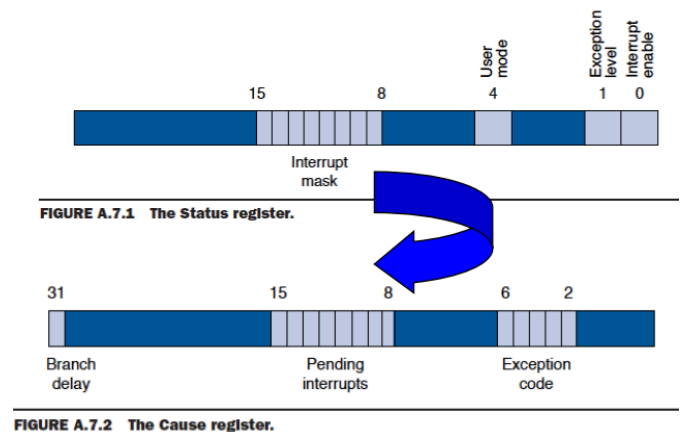


Interrupt mask – **interrupt abilitati**

Un bit per ognuna delle 6 linee delle periferiche e
2 bit per interrupt a livello sw

Interrupt abilitati

- all'interno dell'**interrupt mask** (dello **Status Register**) ciascun bit corrisponde all'**abilitazione** di una **differente linea di interruzione** collegata ad **una o più periferiche**
 - se il **bit** è impostato a **1**, il **corrispondente interrupt** è **abilitato** e la **periferica** che **solleva un'eccezione** può essere "**percepita**"
 - se il **bit** è impostato a **0**, il **corrispondente interrupt** **NON** è **abilitato** e anche se la **linea di interruzione** è **attivata** dalla **periferica**, la **CPU non** genera l'**eccezione**
- quando viene **sollevata un'eccezione** a seguito di un'**interruzione abilitata**, nel **Cause register** viene impostato a **1** il **bit corrispondente all'interruzione** → e i **5 bit dell'exception code** sono impostati a **0**

**Esempio**

Realizzare un programma assembly che stampa tramite syscall, un carattere inserito da tastiera (usando i registri Receiver Control e Receiver Data) attraverso le due tecniche:

- busy wait → gestione da parte del programma
- interrupt → gestione da parte dell'exception handler

→ Esempio busy wait

<pre> .data Messaggio: .ascii "Il tasto premuto e':" #NB questa stringa non e' #terminata con byte 0! Carattere: .asciiz "\n\n" # due a capo (il primo sara' sovrascritto) Fine: .ascii "q" # carattere da premere per terminare .text main: li \$t0, 0xFFFF0000 # Receiver Control li \$t2, 0xFFFF0004 # Receiver Data lb \$t4, Fine # carattere per finire </pre>	<pre> BusyWaitRead: # "busy wait" lw \$t1, 0(\$t0) # Receiver Control Register andi \$t1, \$t1, 0x1 # tiene solo il bit 0 beqz \$t1, BusyWaitRead # se=0, non e' arrivato nulla # se= 1, e' arrivato un carattere lb \$t3, 0(\$t2) # legge byte dal Receiver Data beq \$t3, \$t4, fine # se e' il tasto di fine, esce sb \$t3, Carattere # salva il carattere letto in memoria #sovrascrivendo il primo 'a capo' li \$v0, 4 # stampa il messaggio con la syscall la \$a0, Messaggio # il carattere e' incorporato ! syscall # stampa con syscall j BusyWaitRead fine: li \$v0, 10 #terminazione syscall </pre>
--	--

→ Esempio Interrupt

<pre> # Porzione di codice dell'Exceptions.s per gestione interrupt da Receiver IntKbd: andi \$a0, \$k0, 0x0100 #in k0 era stato copiato Cause # Receiver mask: 0x0100 = 0000 0001 0000 0000 beq \$a0, \$0, EndIntKbd # branch se e' interrupt ma non di Receiver # E' interrupt del Receiver, quindi ... # Stampa le info sull'interrupt li \$v0, 4 la \$a0, __m5_ syscall # print " by Receiver " </pre>	<pre> # DISABILITA Interruzioni dal Receiver (tastiera) lw \$a0, 0xFFFF0000 # Leggi attuale Receiver Control andi \$a0, \$a0, 0xFFFD # Azzera bit 1 = INT enable sw \$a0, 0xFFFF0000 # Scrivi nuovo Receiver Control # DISABILITA Interrupt Receiver nella CPU mfc0 \$a0, \$12 # Leggi il CPU Status Reg. andi \$a0, \$a0, 0xFEFF # Azzera il bit INT del Receiver mtc0 \$a0, \$12 # Scrivi il nuovo CPU Status Reg. # Gestione del char digitato lw \$a0, 0xFFFF0004 # prelevo il carattere dal Data Reg. sw \$a0, BuffKbd # e lo metto in memoria (buffer) </pre>
---	--

EndIntKbd:

ABILITA interrupt della CPU

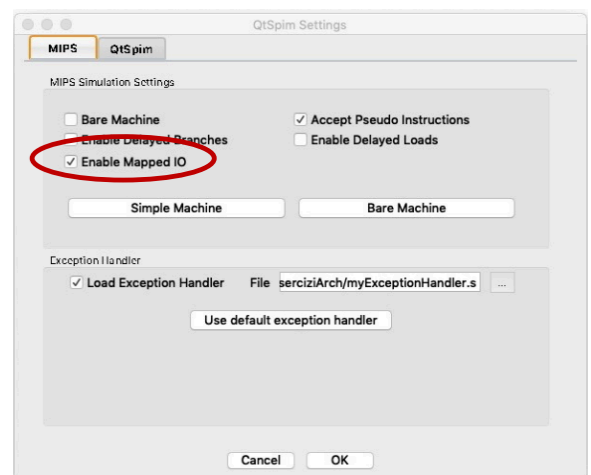
```
mfc0 $t0, $12      # Leggi CPU Status register
ori  $t0, $t0, 1    # abilita in generale gli INT
ori  $t0, $t0, 0x100 # abilita il bit INT del Receiver
mtc0 $t0, $12      # Scrivi nuovo CPU Status register
```

ABILITA Interrupt dal Receiver (tastiera-keyboard)

```
lw  $t1, 0xFFFF0000 # Leggi attuale Receiver Control
ori $t1, $t1, 2      # abilita il bit 1 (= INT enable)
sw  $t1, 0xFFFF0000 # scrivi nuovo Receiver Control
```

→ I/O IN SPIM

- SPIM simula una periferica di I/O (terminale) che permette a un programma MIPS in esecuzione su SPIM di:
 - leggere i caratteri digitati a tastiera
 - scrivere caratteri sulla console (fin. di output)
- nelle impostazioni di SPIM deve essere abilitata l'opzione Enable Mapped I/O (vedi immagine)



SPIM terminal device

- il terminale è composto da due dispositivi **distinti e indipendenti**:
 - Receiver** → legge i caratteri ASCII dalla tastiera
 - Transmitter** → visualizza caratteri ASCII sulla Console
- le unità *Receiver* e *Transmitter* sono completamente **indipendenti**:
 - un tasto premuto sulla tastiera non viene automaticamente mostrato sulla console
 - un programma deve esplicitamente fare “eco” di un carattere letto dal Receiver inviandolo al Transmitter per mostrarlo sulla console

Gestione SPIM terminal device

- Per il controllo del device di I/O da parte di un programma sono utilizzati **4 registri memory-mapped**
- Ogni registro è in realtà un'area di memoria di **32 bit**, ad un indirizzo noto

Receiver Data register (at address 0xffff0004)

- “registro” **read-only**
- cambia quando un *nuovo carattere* è digitato da tastiera

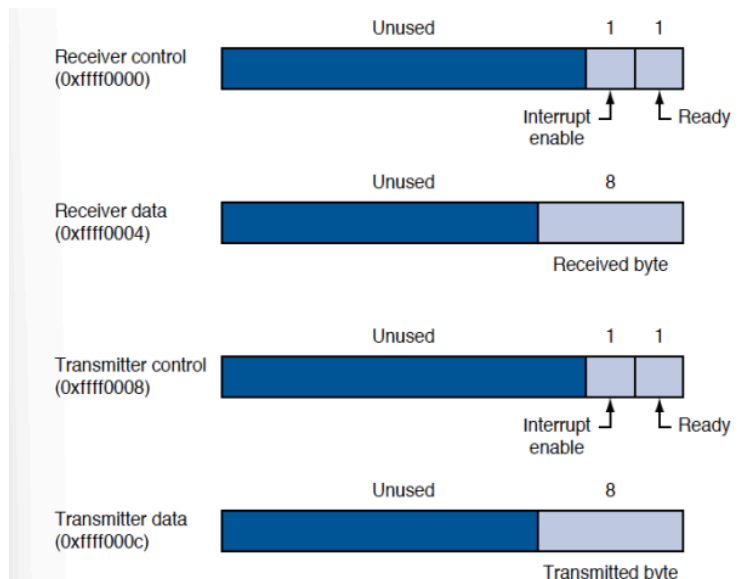
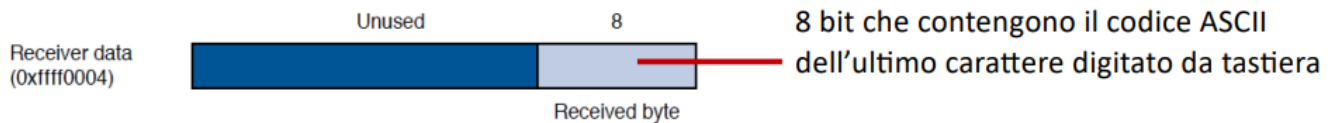


FIGURE A.8.1 The terminal is controlled by four device registers, each of which appears as a memory location at the given address. Only a few bits of these registers are actually used. The others always read as 0s and are ignored on writes.

- il codice ASCII del tasto premuto è scritto negli *8 bit meno significativi*
- il suo contenuto è valido soltanto se **ready bit** del **Receiver Control** è **1** → non è definito se il *ready bit* del *Receiver Control* è **0**
- la sua **lettura** riporta a **0** il *ready bit* del *Receiver Control*



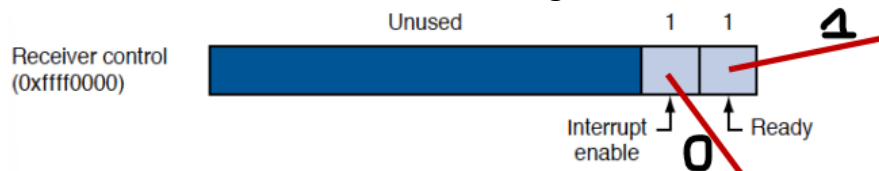
Receiver Data register (at address 0xffff0000)

→ Bit 0 → keyboard ready bit

- è in sola lettura (le scritture vengono ignorate)
- passa da **0** a **1** quando è *digitato un carattere da tastiera*
- **resta a 1**, fintanto che *il carattere non è prelevato*

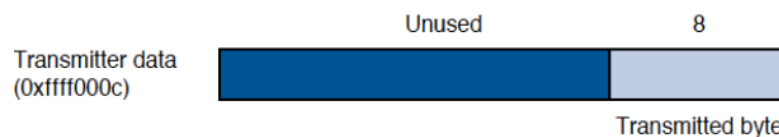
→ Bit 1 → keyboard “interrupt enable” bit

- può essere sia letto che scritto da un programma
- inizialmente è a **0**
- se il programma lo mette a **1**, il terminale **genera un interrupt ogni volta che è digitato un carattere**
- perchè gli interrupt abbiano *effetto sul comportamento del processore*, gli **interrupt** devono anche **essere abilitati** nello **Status register**



Transmitter Data register (at address 0xffff000c)

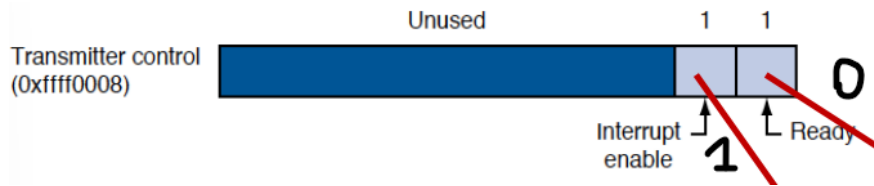
- quando un valore è scritto in questo registro...
 - gli 8 bit **0–7** di questo registro (cioè *codifica ASCII di un carattere*) sono trasmessi alla console
 - il **Ready bit** del registro **Transmitter Control** è messo a **0**
 - Rimarrà a 0 finchè sarà trascorso un tempo sufficiente per *trasmettere il carattere al terminale di output* e poi il **Ready bit tornerà ad essere 1**
- può essere **scritto** quando il **Ready bit** del *registro Transmitter Control* è a **1**
- se **Ready bit = 0** (il transmitter non è pronto), i dati scritti nel Transmitter Data register sono ignorati



Transmitter Control Register (at address 0xffff0008)

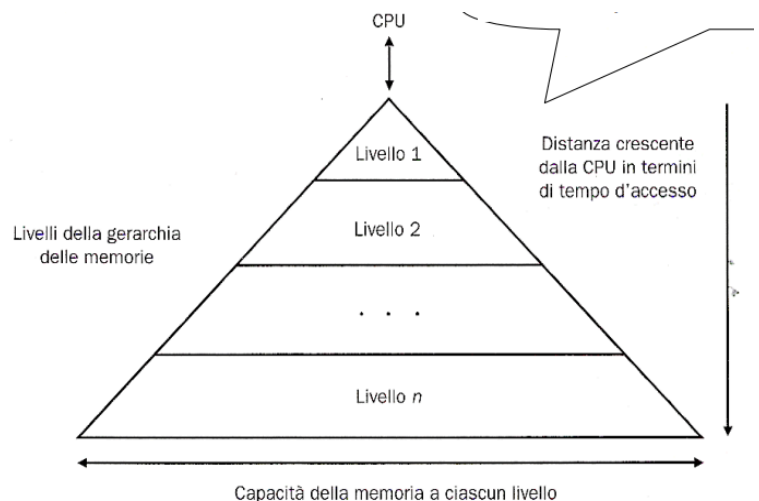
- usati **solo 2 bit**
- comportamento *simile* a Receiver Control register
- Bit 0 → ready bit
 - **read-only**
 - se è a **1**, il **Transmitter** è pronto per accettare un nuovo carattere
 - se è **0**, il **Transmitter** sta ancora scrivendo il carattere precedente
- Bit 1 → interrupt enable bit
 - può essere **letto o scritto**
 - se è a **1**, il *terminale richiede un'interruzione all'hardware* quando il transmitter è pronto per un nuovo carattere → **Ready bit a 1**

- se il **Ready bit** è a **0**, il carattere non sarà visualizzato e andrà perso



GERARCHIE DI MEMORIA

- L'ideale sarebbe avere una memoria illimitata, a basso costo, con alta velocità di accesso e a basso consumo, ma è impossibile avere tutto questo contemporaneamente
- **Premessa** → un programma non accede a tutte le sue istruzioni e a tutti i suoi dati contemporaneamente e con la stessa probabilità
- **Soluzione** → una **gerarchia di memoria** → consiste in un *insieme di livelli di memoria*, ciascuno caratterizzato da una *diversa velocità e dimensione*
- a parità di capacità, le memorie più veloci hanno un **costo più elevato per singolo bit** di quelle più lente
- Al crescere della **distanza** dalla CPU diminuisce il **costo** e aumenta la **capacità di memorizzazione**
- **dati** → sono variabili / costanti
- **informazioni** → ciò a cui accede la CPU attraverso i livelli adiacenti
 - la CPU accede a tali informazioni tramite un **indirizzo** che permette di reperirle nei *livelli di memoria sottostanti*
 - le informazioni sono copiate / passate solo attraverso i **livelli adiacenti di memoria**
- i livelli superiori contengono un sottoinsieme di informazioni disponibili ai livelli inferiori di memoria
- la memoria è **organizzata gerarchicamente** all'interno dell'elaboratore
 - **livelli alti** → memoria *poco capiente, molto veloce, molto costosa*
 - **livelli bassi** → memoria *molto capiente, più lenta, meno costosa*
 - sia HW che SO gestiscono la gerarchia dei dati a livello di memoria per accedere all'informazioni nel modo più veloce possibile, quindi sfruttando la gerarchia
- la **memoria interna** alla CPU è costituita dai registri ed è caratterizzata da **alta velocità** (comparabile a quella del *processore*) e **limitate dimensioni** (capacità di memorizzazione dei registri limitata)
- la **memoria centrale** (principale) è caratterizzata da **dimensioni molto maggiori** (*grandi*) della memoria interna alla CPU, ma con **tempi di accesso più elevati**
 - è accessibile in modo **diretto** tramite **indirizzi**
 - nei sistemi attuali un livello di memorie cache (*dispositivo HW*) è stato *inserito tra CPU e memorie centrali*
 - la memoria cache è uno *strato nella gerarchia* tra la CPU e la memoria centrale
- le **memorie secondarie** sono ad **alta capacità, bassi costi e non volatili**



PRINCIPIO DI LOCALITA'

- un programma, in un certo istante di tempo, accede soltanto a una porzione relativamente piccola del suo spazio di indirizzamento
- rappresenta la base del comportamento dei programmi in un calcolatore
- la **località** indica il **modo** in cui si **accede** ad un'informazione per *ottimizzarla*
- Esistono due principi di località:
 - 1) **Principio di località temporale** → quando si fa riferimento a un **elemento** c'è la tendenza (necessità) a fare riferimento allo **stesso elemento** dopo poco tempo
 - 2) **Principio di località spaziale** → quando si fa riferimento a un **elemento** c'è la tendenza (necessità) a fare riferimento poco dopo ad **altri elementi** che hanno l'**indirizzo vicino** ad esso
 - elementi *vicini* da un punto di vista di indirizzamento → accesso alle *componenti adiacenti*
- ★ i programmi NON vedono la gerarchia ma **referenziano i dati** come se fossero sempre in memoria centrale
- il principio di località viene sfruttato interrogando la memoria in modo gerarchico
- **velocità** → più è veloce, più è vicina al processore, più è costosa
- **dimensione** → più è grande, più è lontana dal processore, meno è costosa
 - più tempo per trasportare il dato al processore
- un livello di memoria più vicino al processore contiene un sottoinsieme di dati memorizzati in ogni **livello sottostante** e *tutti i dati si trovano nel livello più basso*

Differenza tra principio di località spaziale e località temporale

La **località temporale** dice che è probabile che un oggetto a cui si è fatto riferimento, venga nuovamente richiesto in tempo breve.

La **località spaziale** dice che è probabile che gli oggetti che si trovano vicino ad un oggetto a cui si è fatto riferimento vengano richiesti in tempi brevi

DEFINIZIONI

- **blocco / linea** → la più piccola quantità di informazione che può essere presente / assente in una *gerarchia di memoria*
- **Hit (successo nell'accesso)** → l'*informazione richiesta* dal processore si trova in uno dei blocchi nel livello superiore di memoria
- **Miss (fallimento nell'accesso)** → il dato non è presente nel *livello immediatamente inferiore* ed occorre accedere al *livello più distante*
- **Hit rate (frequenza di hit)** → *frazione degli accessi alla memoria* nei quali l'informazione richiesta è *stata trovata* nel livello superiore di memoria → quante volte la richiesta è andata a buon fine

$$\text{frequenza di hit} = \frac{\text{accessi risolti con successo}}{\text{numero totale di accessi}}$$

- **Miss rate (frequenza di miss)** → *frazione degli accessi alla memoria* nei quali l'informazione richiesta *NON è stata trovata* nel livello superiore di memoria (**1 – HitRate**)

$$\text{frequenza di miss} = \frac{\text{accessi risolti senza successo}}{\text{numero totale di accessi}}$$

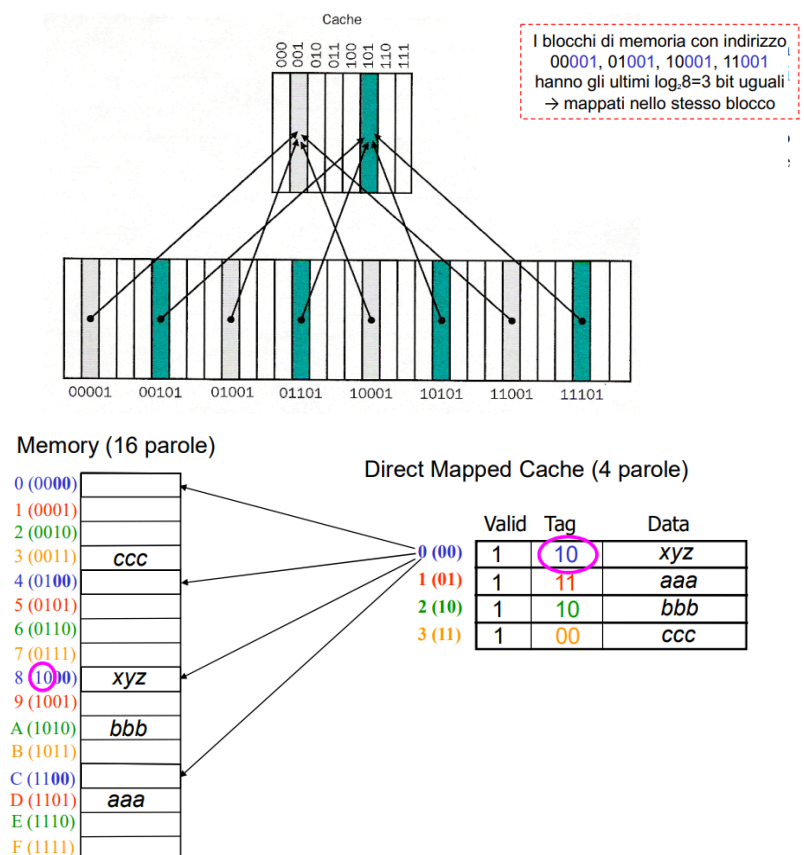
- **Tempo di hit** → tempo di accesso al livello superiore della memoria → a cui si richiede l'informazione
 - comprende anche il tempo necessario a stabilire se il tempo di accesso si risolve in un successo o un fallimento
- **Tempo di miss** → il tempo necessario a sostituire un blocco del *livello superiore* con un nuovo blocco dal *livello inferiore* della gerarchia, e **trasferire i dati** di questo blocco al processore
 - 1) accedere all'informazione
 - 2) verificare se l'informazione è presente
 - 3) sostituire un blocco del livello superiore
- ★ **Nota Bene** → una gerarchia di memoria può essere composta da più livelli, ma i **dati** vengono **trasferiti solo tra due livelli vicini**

CACHES

- **cache** → livello della memoria gerarchica che si trova tra il **processore** e la **memoria principale**
- la memoria cache e il suo utilizzo sono generalmente *trasparenti* al programmatore (quindi nascosta, il programmatore non può vederla)
- l'**algoritmo di caching** si basa sui principi di **località spaziale** e **temporale**
 - mantiene i *dati richiesti recentemente* “vicino” alla CPU (**località temporale**)
 - *muove blocchi contigui di memoria* che contengono il *dato richiesto* (**località spaziale**)
- tecniche di caching:
 - 1) direct mapping – mappatura diretta
 - 2) fully associative
 - 3) set associative

1) DIRECT MAPPING – mappatura diretta

- a *ciascun blocco della memoria* corrisponde una **specificata locazione** nella *cache*
- la corrispondenza è tra **informazioni nella memoria centrale** e **indirizzi nella cache**
- a ciascun **blocco di memoria** è associata una **linea di cache**
- **diversi blocchi di memoria** possono corrispondere alla **stessa linea di cache**
- più *indirizzi in memoria* sono referenziati nella *stessa linea di cache* → stessi blocchi **condividono i bit meno significativi**
- occorre risolvere l'*ambiguità di individuazione del blocco* con delle strategie



- La cache a **mappatura diretta** è la *forma più semplice di cache*
- associa una sola locazione della cache a ogni **parola** della memoria definendo una **corrispondenza** tra l'**indirizzo in memoria** della parola e la **locazione della cache**
- ★ !! per trovare il blocco che corrisponde ad un indirizzo della memoria principale:
(**indirizzo del blocco**) **modulo** (**numero di blocchi nella cache**)
- indirizzo:
 - **tag** → (*etichetta*) contiene informazioni necessarie a verificare se una parola della cache **corrisponde** o meno alla parola cercata
 - **indice** → utilizzato per selezionare il blocco della cache
 - **offset** → bit necessari per selezionare il byte richiesto nella parola
- ★ se il *numero di blocchi* nella cache è una *potenza di 2*, la **posizione** corrispondente della parola in cache è data dai **$\log_2$ (numero di elementi nella cache)** numero di **bits meno significativi** dell'indirizzo in memoria principale

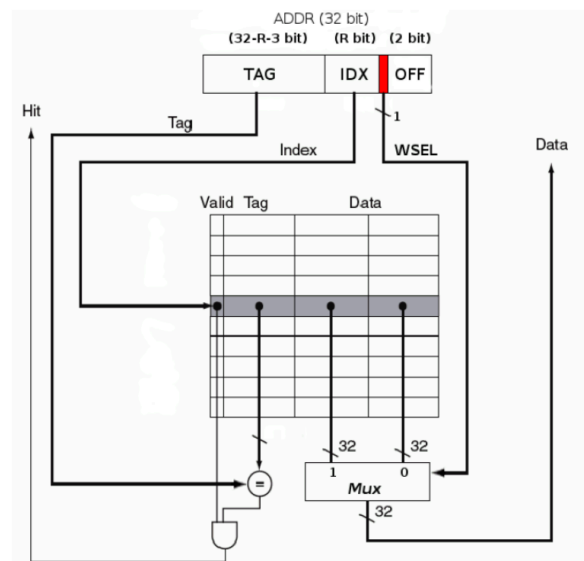
2 – Word Direct Mapped Cache in figura:

2 problemi:

- 1) disambiguare il *reperimento delle informazioni* in memoria
- 2) *reperire le informazioni* in memoria

2 confronti:

- 1) sul *bit di validità* (0)
 - dimostra che il dato è **valido**
 - quando **non** è valido?
 - 1) all'**avvio di un elaboratore** il bit viene impostato a zero di default
 - 2) quando il dato **non è pronto** per essere *preso*
- 2) sul **TAG**, se i **due TAG** sono diversi le parole *non combaciano*



Multiplexor dell'immagine → selettore di un MUX che mi dice quale **informazione** è quella *ricercata*, proveniente da un **blocco di cache** contenente *più informazioni* (in questo caso sono referenziate 2 parole in 1 stessa linea di cache)

ESEMPIO – Si consideri la seguente situazione:

- indirizzo su **32 bit**
- *cache a mappatura diretta*
- la *dimensione* della **cache** è di **2^n blocchi**, dove *n bit* vengono usati per l'**indice**
- la *dimensione* del **blocco** della cache è di **2^m parole**, ossia **2^{m+2} byte**, per cui *m bit* vengono usati per individuare una **parola all'interno di un blocco**, mentre **2 bit** per individuare un **byte all'interno di una parola**
- la dimensione del campo TAG è → **$32 - (n + m + 2)$**
- il **numero totale di bit** contenuti in una cache a mappatura diretta è:

$$2^n * (\text{dimensione_blocco} + \text{dimensione_TAG} + \text{bit_validità})$$

$$2^n * (2^m * 32 + (32 - (n+m+2)) + 1)$$

❖ *Osservazione* → in realtà, anche se la *dimensione reale della cache* è composta dai **dati**, **tag** e **bit di validità**, per convenzione si considera **solo** la *dimensione* dei **dati**

ESERCIZIO:

Testo: Da quanti bit è costituita una cache a mappatura diretta contenente 16KB di dati e avente blocchi da 4 parole, ipotizzando un indirizzo di memoria su 32 bit?	Soluzione:
--	-------------------

Mappatura di un indirizzo:

Consideriamo ora una cache con 64 blocchi di 16 byte ciascuno.

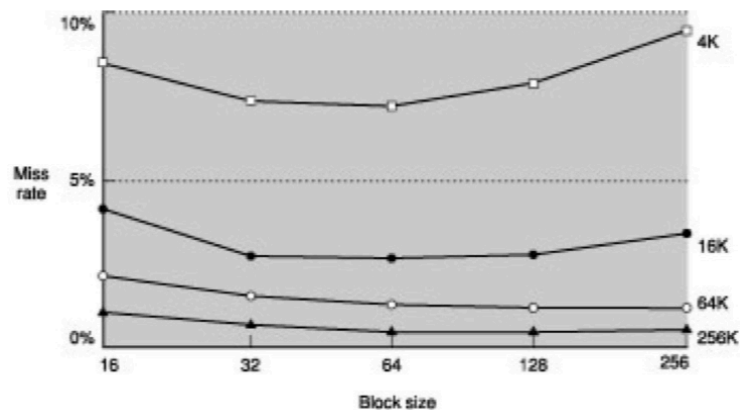
L'indirizzo 1200 (in byte) può essere ricavato come:

(indirizzo del blocco) modulo (numero blocchi nella cache)

- indirizzo del blocco → $1200/16 = 75$
 - ◆ **indirizzo del blocco = indirizzo del dato in byte / ampiezza del blocco in byte = 75**
- modulo → mod
- numero blocchi nella cache → 64
- $75 \text{ mod } 64 = 11$ → **risultato**

DIMENSIONI DEL BLOCCO DI CACHE

- la cache referencia i **blocchi**, NON è fatta di blocchi
- la **dimensione del blocco** di cache è in stretta relazione con la **frequenza di miss**
- la **frequenza di HIT / MISS** è legata alla *dimensione del blocco* nella cache
- quando abbiamo **blocchi molto grandi** possiamo sfruttare il principio di **località spaziale** senza avere la necessità di *recuperare l'informazione* in **livelli sottostanti** → in quanto si rischia un **alto miss rate**
- quando la *dimensione del blocco* è molto più grande della *dimensione della cache* c'è un'alta probabilità di generare un **miss**
- se la cache è molto grande, **aumenta il costo** per reperire una singola informazione ma diminuisce il miss rate e aumenta l'hit rate

**SCELTA DELLA DIMENSIONE DEL BLOCCO**

- in generale, un'ampia **dimensione per il blocco** permette di sfruttare la **località spaziale**
 - blocchi di grossa dimensione comportano maggiori **miss penalty**
 - è necessario **più tempo** per trasferire il blocco
 - se la **dimensione del blocco** è troppo grossa rispetto alla dimensione della cache, il **miss rate aumenta**
 - se la dimensione del blocco da sostituire è maggiore di quella della cache, allora il **numero di blocchi** nella cache è **insufficiente**

- **MISS PENALTY** → differenza tra il tempo di accesso al livello inferiore e il tempo di accesso alla cache

2) FULLY ASSOCIATIVE

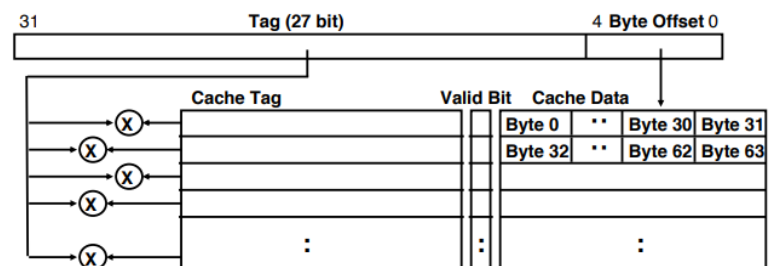
- **ogni blocco** può essere collocato in **qualsiasi locazione** della cache (posizione nella cache) → ricerca sequenziale
- per ricercare un blocco nella cache è necessario **cercarlo in tutte le linee** della cache
- la ricerca sequenziale è troppo lenta → ricerca in parallelo (soluzione *molto costosa a livello HW*)
- tipologia di cache più costosa
- nella rappresentazione non dobbiamo individuare l'informazione giusta nello stesso blocco come nella mappatura diretta
- **ricerca sequenziale** → non abbiamo riferimenti in memoria, le informazioni potrebbero essere in qualsunque locazione, quindi per trovare l'informazione dobbiamo spendere molto a livello HW → il **TAG** è confrontato con il TAG di ogni linea di cache fino a trovare un match
- **Tag** → elemento che ci permette di capire se l'**informazione trovata è quella ricercata**
- **associazione 1 a 1** → 1 linea/blocco di cache referenzia 1 linea di memoria centrale

➤ nella cache *fully associative* **non** viene impiegata *alcuna indicizzazione*

➤ vengono confrontati in parallelo i tag di tutti i blocchi appartenenti alla cache

→ **esempio:**

- ◆ indirizzo da 32 bit
- ◆ dimensione di blocco = 32 byte → tag da 27 bit
- ◆ N blocchi → necessari N comparatori da 27 bit



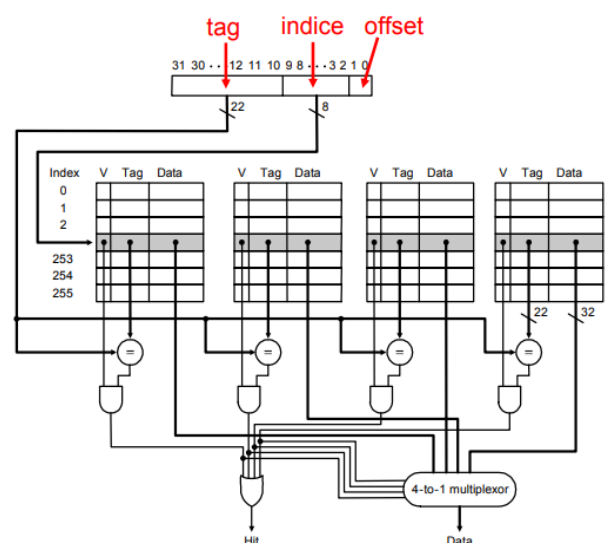
3) SET ASSOCIATIVE

- soluzione intermedia tra direct mapped e fully associative
- ciascun blocco della memoria ha a disposizione un **numero fisso** (≥ 2) di **locazioni** in cache definiti come **set**
- **set associative** → ossia cache **organizzata in insiemi**, dove un indice che *indica più blocchi* può referenziare un solo set e ogni set è composto da più righe
- ogni indirizzo di memoria corrisponde ad **un unico e un solo set di righe** della cache

➤ i blocchi appartenenti alla cache sono raggruppati in **set**

➤ in una **cache set associative ad N vie** (**N-way set associative**) ogni set raggruppa N blocchi

Cache 4-Way Set Associative:



- ogni indirizzo di memoria corrisponde ad un unico set della cache (accesso tramite indice) e può essere ospitato in un blocco qualunque appartenente a quel set
- stabilito il **set**, per determinare se un certo indirizzo è *presente* in un blocco è necessario confrontare in parallelo i tag di tutti i blocchi

Immagine:

- ➔ **tag** → viene fatto su tutti i blocchi di uno stesso set indicato dall'**indice**
- ➔ **indice** → indirizza verso la linea di indice della cache
- ➔ questo è un set formato da 4 blocchi
- ➔ **= / AND** → porte che gestiscono il *match* tra i tag e il bit di validità
- ➔ il *risultato* di queste porte fluisce in un **OR** e da questo il dato che viene fornito in *output* può essere un **HIT** o un **MISS** → verifica se il *dato* è *presente*
- ➔ **MUX** → ho *più componentistica* perchè devo dire se il *dato* è *presente* e devo fornire un dato specifico contenuto in uno dei blocchi del set → ha più blocchi quindi con un MUX viene indicato da quale blocco *proviene* l'informazione richiesta

SVANTAGGI DELLA CACHE SET ASSOCIATIVE

- cache **N-way set associative** a confronto con cache ad **accesso diretto**:
 - 1) N comparatori invece di **1**
 - 2) **ulteriore ritardo** fornito dal *multiplexor*
 - 3) il *blocco* è disponibile dopo la decisione **HIT / MISS** e la **selezione del set**
- In una *cache ad accesso diretto*, il blocco è disponibile prima della decisione **HIT/MISS**

CAMPI DELLA CACHE

- ➔ **tag** → **etichetta**, contiene informazioni necessarie a *verificare* se una parola della cache **corrisponde** o meno alla *parola cercata*
 - ◆ consente di individuare in modo **univoco** il blocco in memoria che è stato *mappato* nella linea di cache → contiene informazioni necessarie a verificare se una parola della cache *corrisponde o meno* alla parola cercata
- ➔ **indice** → utilizzato per **selezionare** il blocco della cache

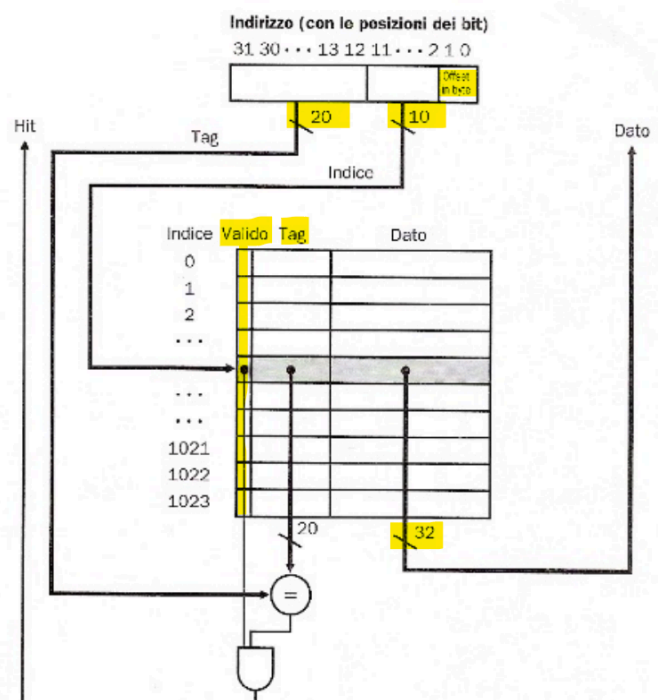
Recup campi della cache:

- i **bit meno significativi** dell'indirizzo del dato richiesto indicano la **posizione** (**indice**) del dato nella cache
- Osservazione → più blocchi di memoria sono destinati alla stessa posizione nella cache! Come riconoscere il dato?
- I **bit più significativi** dell'indirizzo del dato richiesto rappresentano il **TAG**
- ES. indirizzo : 00101 → **Tag** : 00
indice : 101

Cache a mappatura diretta:

In una **cache ad indirizzamento diretto** ogni linea di cache include:

- ➔ **bit di validità** → un **campo** che indica se il blocco di memoria associato contiene un **dato valido** (*true/false*)



◆ *all'avvio tutte le linee sono NON VALIDE* → **compulsory miss**

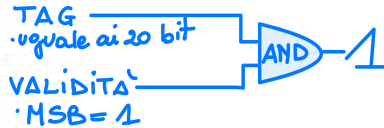
→ il **blocco di dati** vero e proprio, formato da *una o più parole*

→ **2 bit di offset** → mi permette di individuare il *byte all'interno della parola*

→ \ 10 → bit usati per l'**indirizzamento** nella cache

→ \ 20 → sono i **20 bit più significativi** → se sono uguali al **TAG** della cache e se il *bit più significativo* è 1 il dato è **presente e valido**

◆ se il **MSB** è 0 allora il dato **NON** è valido



→ \ 32 → *uscita dal processore* se ho generato un **HIT**

GESTIONE DI CACHE HIT E CACHE MISS

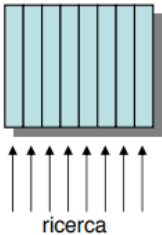
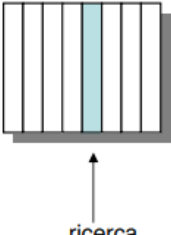
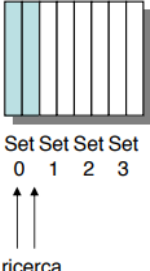
In caso di **HIT**:

- il processo **non** viene **interrotto** dal processore
- **accesso al dato** della *cache dati*
- **accesso all'istruzione** della *cache istruzioni*

In caso di **MISS**:

- **stallo del processore** (come nel pipelining) in attesa di ricevere l'elemento della memoria
- **invio dell'indirizzo** al *controller della memoria* (simile ad un DMAC, identificato anche come MMU – Memory Management Unit)
- **reperimento** dell'*elemento dalla memoria*
- **caricamento** dell'*elemento in cache*
- **ripresa dell'esecuzione**

CONFRONTO TRA LE TECNICHE DI INDIRIZZAMENTO

FULLY ASSOCIATIVE	DIRECT MAPPED	SET ASSOCIATIVE																																																												
il blocco 12 può essere disposto ovunque	il blocco 12 può essere disposto solo sul blocco 4 (12 mod 8)	il blocco 12 può essere disposto ovunque nel set 0 (12 mod 4)																																																												
Blocco 0 1 2 3 4 5 6 7 	Blocco 0 1 2 3 4 5 6 7 	Blocco 0 1 2 3 4 5 6 7 																																																												
Blocco  Memoria di livello inferiore <table><tr><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>1</td><td>1</td><td>1</td><td>1</td><td>1</td><td>1</td><td>1</td><td>1</td><td>2</td><td>2</td><td>2</td><td>2</td><td>2</td><td>2</td><td>2</td><td>2</td><td>3</td><td>3</td></tr><tr><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td><td>7</td><td>8</td><td>9</td><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td><td>7</td><td>8</td><td>9</td><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td><td>7</td><td>8</td><td>9</td><td>0</td><td>1</td></tr></table>			0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1	2	2	2	2	2	2	2	2	3	3	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9	0	1
0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1	2	2	2	2	2	2	2	2	3	3																																			
0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9	0	1																															

DIMENSIONE DEL TAG E ASSOCIATIVITA'

- aumentando il *grado di associatività*
 - *aumenta* il numero dei comparatori e il numero di bit per il tag
- esempio:
 - cache con 4K blocchi, blocco di 4 parole, indirizzo a 32 bit
 - $r = \log_2(4 \cdot 4) = 4 \rightarrow n - r = (32 - 4) = 28$ bit per tag e indice
 - ◆ cache ad **indirizzamento diretto**
 - $s = \log_2(4K) = 12 \rightarrow 2^2 \cdot 2^{10} \rightarrow 2^{12} \rightarrow 12 \rightarrow$ posizione della parola in cache (**indice**)
 - bit di tag totali = $(28 - 12) \cdot 4K = 64K$
 - ◆ Cache **set-associative a 2 vie**
 - $s = \log_2(4K/2) = 11$
 - bit di tag totali = $(28 - 11) \cdot 2 \cdot 2K = 68K$
 - ◆ cache **set associative a 4 vie**
 - $s = \log_2(4K/4) = 10$
 - bit di tag totali = $(28 - 10) \cdot 4 \cdot 1K = 72K$
 - ◆ cache **completamente associativa**
 - $s = 0$
 - bit di tag totali = $28 \cdot 4K \cdot 1 = 112K$

INCREMENTO DELL'ASSOCIATIVITA'

- Principale *vantaggio* → *diminuzione del miss rate*
- Principale *svantaggio* → *maggior costo implementativo*
- la scelta tra cache ad *indirizzamento diretto*, *set-associative* e *completamente associativa* dipende dal costo dell'associatività rispetto alla riduzione del miss rate

GESTIONE DELLA CACHE

Quattro decisioni da prendere:

1) Dove posizionare un blocco? 2) Come reperire un blocco?	Tecnica di indirizzamento
3) Quale blocco sostituire in corrispondenza di un miss? → se il dato non è presente in cache devo creare spazio per inserire il dato preso dalla memoria	Algoritmo di sostituzione
4) Come gestire un'operazione di scrittura?	Strategia di aggiornamento

ALGORITMI PER LA SOSTITUZIONE DEI BLOCCHI

Nella cache ad **accesso diretto**:

- se il blocco di memoria è **mappato** in una linea di cache già occupata (**conflict miss**), si elimina il contenuto precedente della linea e si **rimpiazza** con un **nuovo blocco**
- ❖ Quale blocco sostituire in caso di (**capacity**) **miss** nella cache *set- o fully- associative*?
 - in caso di **cache fully-associative** → ogni blocco è un potenziale candidato per la sostituzione
 - in caso di **cache set-associative a N vie** → bisogna scegliere tra gli *N blocchi del set*

Esistono **3 politiche di sostituzione dei blocchi**:

- 1) Politica di sostituzione **Random** – scelta casuale
 - vantaggio → *facilmente implementabile* a livello hardware

→ usata quando vi è un livello di associatività molto alta nella cache fully associative

2) Politica di sostituzione **Least Recently Used (LRU)**

- *least recently used* → il più vecchio → il blocco con minor probabilità di essere usato verrà sostituito
- **LRU** sfrutta al meglio il **principio di località temporale**, infatti sfruttando tale principio il blocco sostituito è quello che **non si utilizza da più tempo**
- ad ogni blocco si associa un **contatore all'indietro**, che viene portato al valore massimo in caso di accesso e decrementato di 1 ogni volta che si accede ad un altro blocco
 - ◆ *contatore all'indietro* → necessario tenere *traccia degli HIT e dei MISS* per ogni singolo blocco per capire quale blocco è referenziato più frequentemente

3) Politica di sostituzione **First In First Out (FIFO)**

- si approssima la strategia LRU selezionando il **blocco più vecchio** anziché quello *non usato da più tempo*

STRATEGIA DI AGGIORNAMENTO

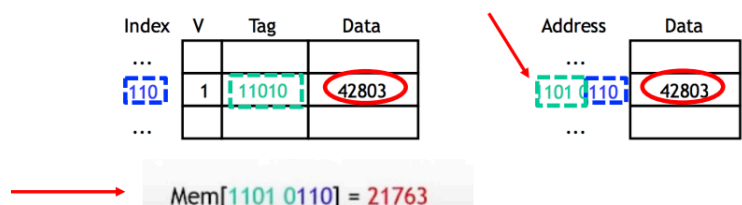
GESTIONE DEI MISS IN LETTURA

- Se un blocco **non è presente** in cache bisogna mettere in stallo l'intera CPU
 - perchè la CPU ci chiede un dato non presente in cache quindi dobbiamo **bloccare il processo** che lo ha richiesto finchè il dato non viene reperito e non sarà **presente in cache**
- in generale al verificarsi di un miss nella cache delle istruzioni sono necessari i seguenti passi:
 - 1) inviare **PC - 4** (*uscita ALU*) alla memoria → valore *originale* del **PC**
 - 2) *lettura dalla memoria* → **miss nella cache**
 - *ricerca del dato* in memoria nel *livello inferiore* → mettere il *dato in cache* → *bit di validità* *settato a 1* → viene passato il *tag* e *copiato il dato*
 - 3) **scrittura nella cache** (dato, tag e bit di validità)
 - 4) **riavviare l'esecuzione dell'istruzione** che ha causato il miss
- se il **miss** non riguarda le *istruzioni* ma riguarda i **dati**, si può continuare a *reperire le informazioni legate a quel dato* e andare a **bloccare la CPU** quando si dovrà accedere a quel dato → **stalli in tempi diversi**

ACCESSO IN SCRITTURA

- il trasferimento richiede *del tempo*
- le **scritture** sono *meno frequenti* delle *letture* → se si sfrutta in modo efficiente il **principio di località temporale** avremo più letture (**HIT**) che scritture (**MISS**)

- ★ **scrivere un dato** nella *cache* significa creare un'**incoerenza** se non si aggiornano i livelli inferiori della gerarchia di memorie
- tale **AGGIORNAMENTO** richiede lo **stallo della CPU**



- esistono due tecniche risolutive per scrivere il dato nella cache:
 - 1) **Write-through**
 - 2) **Write-back**
 - 3) utilizzo di un **write buffer**

1) WRITE-THROUGH – Prima strategia.

- **write-through** → i dati sono **scritti contemporaneamente** nel blocco della cache e nel blocco del livello inferiore
- vantaggi
 - 1) è la soluzione più semplice da implementare
 - 2) si mantiene la coerenza delle informazioni nella gerarchia di memorie
- svantaggi
 - 1) le operazioni di scrittura vengono effettuate alla velocità della memoria di livello inferiore
→ di conseguenza **diminuiscono le prestazioni**
 - 2) aumenta il traffico sul bus di sistema

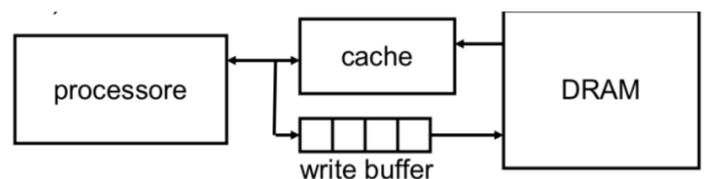
2) WRITE BACK

- **write-back** (o *copy back*) → i dati sono scritti **solo** nel blocco della cache
- il blocco modificato viene *scritto* nel livello inferiore della gerarchia **solo** quando deve **essere sostituito**
- subito dopo la scrittura, cache e memoria di livello inferiore sono ***inconsistenti*** e viene mantenuto un **dirty bit**
- un **bit specifico** segnala se il dato è stato modificato (**dirty**) oppure se il dato ***non*** è stato modificato (**clean**)
- vantaggi
 - 1) le scritture avvengono alla velocità della cache
 - 2) scritture successive sullo stesso blocco alterano solo la cache
- svantaggi
 - 1) ogni sostituzione del blocco può provocare un trasferimento in memoria

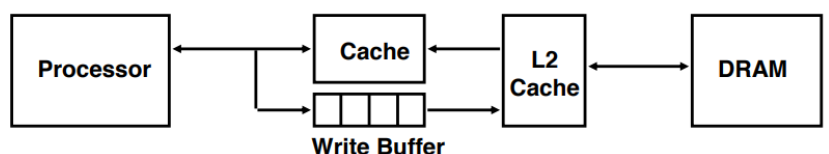
3) WRITE-THROUGH CON WRITE BUFFER

- **write buffer** interposto tra la cache e la memoria di livello inferiore
- il processore scrive i dati sia nella cache sia nel **write buffer**
- il **controller** della memoria scrive il contenuto del write buffer in memoria in modo **asincrono**
- potrebbero essere richieste *frequenti scritture* quindi il write buffer potrebbe andare in **overflow**
 - 1) si può *tornare* alla **write-back**
 - 2) si possono usare **2 livelli di cache**
- il write buffer è gestito in **modalità FIFO**
 - il numero tipico di elementi del buffer è **4**
 - efficiente se la frequenza di scrittura $\ll 1/\text{write cycle}$ della DRAM (memoria)
 - altrimenti il **buffer** può andare in **saturazione** e il processore deve aspettare che le scritture giungano a completamento (**write stall**)

- il **processore** scrive i dati sia nella cache che nel **write buffer**



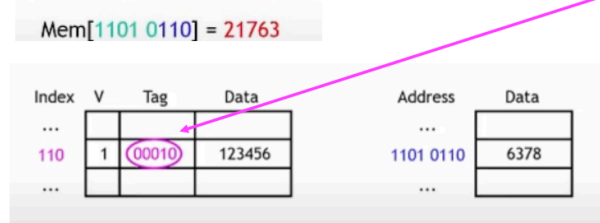
- **cache di secondo livello** per ovviare al *problema* dell'**overflow** del write buffer



4) WRITE MISS

- un secondo scenario di **inconsistenza** potrebbe accadere se cerchiamo di scrivere un indirizzo che **non** è (*ancora*) contenuto in cache, in questo caso si ha un **write miss**
- le scritture possono introdurre un write miss

Immaginiamo di voler memorizzare 21763 all'indirizzo 11010110



- soluzioni possibili:
 - 1) **write allocate** → il *blocco* viene *caricato in cache* e si *effettua la scrittura*
 - 2) **no-write allocate** → il *blocco* viene *scritto direttamente* nella *memoria di livello inferiore*, *senza* essere *direttamente trasferito in cache*
- tipicamente abbiamo le seguenti combinazioni:
 - 1) **write-back** con **write allocate**
 - 2) **write-through** con **write-not allocate**

1. OSSERVAZIONE

- relazione tra probabilità di miss e dimensioni cache/memoria
- una cache con **blocchi di dimensioni maggiori** sfrutta maggiormente la **località spaziale** diminuendo la frequenza di miss
- la **frequenza di miss** torna a *crescere* se la dimensione dei *blocchi* diventa **troppo grande** rispetto alla *dimensione della cache*
- quindi i blocchi vengono scaricati dalla cache *prima* ancora che *molti dati* in essi contenuti siano utilizzati
- quindi la **località spaziale** tra le **parole** di un *blocco* diminuisce, e il *miglioramento* legato alla **frequenza di miss** si riduce
- inoltre, cresce anche il **costo di una miss** → la penalità di una *miss* è determinata dal tempo necessario a prelevare un blocco dal livello sottostante e a scriverlo nella cache
- il **tempo** per *prelevare un blocco* è dato dalla **somma** tra:
 - 1) la latenza per ottenere la prima parola del blocco
 - 2) il tempo di trasferimento del resto del blocco
- il **tempo di miss** comprende:
 - 1) **verifica dell'informazione**
 - 2) **tempo di recupero** del dato nella memoria che si sta referenziando
 - 3) **tempo di trasferimento**